SIGN-LINK

A PROJECT REPORT (PHASE-2)

by

SHALIN PHILIP (VJC22CSD058)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND DESIGN

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN
VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM
MARCH 2026

SIGN-LINK

A PROJECT REPORT (PHASE-2)

by

SHALIN PHILIP (VJC22CSD058)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND DESIGN

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

under the guidance

of

Dr. Peter Reji Ramanatt

Associate Professor, CSD Dept.

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

MARCH 2026

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

Department of Computer Science and Design

Vision

Moulding socially dynamite Computer Science and Design engineers capable of driving technological advancements

Mission

1. To provide comprehensive education that combines technical knowledge, creativity and critical thinking through computer science principles and design methodologies.

2. Promote research and industry collaboration to empower our graduates for addressing real world challenges.

3. To cultivate ethical values, social responsibility and a commitment to lifelong learning among students.

Program Educational Objectives

Our Graduates

1. Shall have strong foundation in theoretical and practical aspects of Computer Science and Design principles to develop innovative solutions.

2. Shall possess effective communication, teamwork and leadership skills to collaborate with diverse stakeholders and to practice in a corporate firm.

3. Shall have potential to become entrepreneurs, innovators and pursue research in Computer Science and Design or related fields.

4. Shall have a strong sense of social awareness and stay updated with evolving design technologies.

Program Outcomes

1. **Engineering knowledge**:Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis**:Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences

3. **Design / development of solutions**:Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety and the cultural, societal and environmental considerations.

4. **Conduct investigations of complex problems**:Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of the information to provide valid conclusions.

5. **Modern tool usage**:Create, select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society:**Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability:**Understand the impact of the professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of and need for sustainable development.

8. **Ethics:**Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice

9. **Individual and team work:**Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary settings

10. **Communication:**Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and unread in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning**:Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes

1. Ability to integrate theory and practice to design software applications.
2. Able to apply computer science skills and design tools to analyse, design and model composite systems.
3. Ability to design and manage projects to develop a career in a related industry.

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

BONAFIDE CERTIFICATE

This is to certify that the Project Report (Phase-2) entitled **" SIGN-LINK"** is a bonafide record of the work done by **SHALIN PHILIP (VJC22CSD058)** in partial fulfillment of the requirements for the award of the **Degree of Bachelor of Technology** in **Computer Science and Design** of **APJ Abdul Kalam Technological University.**

Internal Supervisor **External Supervisor**

Project Coordinator **Head of Department**

ACKNOWLEDGEMENT

First and foremost, I thank **God Almighty** for His divine grace and blessings in making all these possible. May He continue to lead me in the years. It is my privilege to render my heartfelt thanks to my most beloved Manager **Msgr. Dr. PIUS MALEKANDATHIL**, my Director **Rev. Fr. PAUL PARATHAZHAM** and my Principal **Dr. K K RAJAN** for providing me the opportunity to do this Project Report (Phase 2) during the seventh semester of my B.Tech degree course. I am deeply thankful to my Head of the Department , **Mrs. SABITHA RAJU** for her support and encouragement. I would like to express my sincere gratitude and heartfelt thanks to my Project Guide **Dr. PETER REJI RAMANATT**, Associate Professor, Department of Computer Science and Design for his motivation, assistance and help for the project. I also express sincere thanks to my Project Coordinator **Mrs. ANJU MARKOSE**, Assistant Professor, Department of Computer Science and Design for her guidance and support. I also thank all the staff members of the Computer Science And Design Department for providing their assistance and support. Last, but not the least, I thank all my friends and family for their valuable feedback from time to time.

DECLARATION

I undersigned hereby declare that the project report "SIGN-LINK", submitted for partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology of the APJ Abdul Kalam Technological University is a bonafide work done by myself under the supervision of Dr. Peter Reji Ramanatt. This submission represents ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or formed the basis for the award of any degree, diploma or similar title of any other University.

Place : Vazhakulam

Date : 31-03-2026

SHALIN PHILIP

ABSTRACT

The ”SIGN-LINK” is an innovative mobile application designed to bridge the communication gap between deaf-mute and normal-speaking individuals through advanced real-time translation technologies. The app enables users to select their communication type-impaired or normal-upon entry. During video calls, speech from the normal user is instantly converted into live text captions or sign language animations for the impaired user. Conversely, when the impaired user communicates using sign language, the app translates their gestures into text and voice output for the other participant. Additionally, SIGN-LINK includes a live translation feature that allows users to interpret sign language in real-world situations using their device's camera, providing instant voice or text translation. By integrating speech recognition, gesture detection, and artificial intelligence-based translation systems, the app ensures seamless, inclusive, and effective communication. Prioritizing accessibility and inclusivity, SIGN-LINK redefines interpersonal interaction, fostering understanding and connection between speech-impaired and normal-speaking communities.

Contents

List of Figures

List of Abbreviations

AI	Artificial Intelligence
ML	Machine Learning
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
SLR	Sign Language Recognition
ASR	Automatic Speech Recognition
NFC	Near Field Communication
API	Application Programming Interface
UI	User Interface
UX	User Experience
GPU	Graphics Processing Unit
QoE	Quality of Experience

Chapter 1

INTRODUCTION

In today's digital world, communication plays a vital role in connecting people, but individuals with hearing or speech impairments often face major challenges in interacting with others. Traditional communication methods such as texting or writing notes are time-consuming and lack the expressiveness of speech and gesture. With the advancement of artificial intelligence and computer vision, technology now offers the potential to bridge this communication gap and make real-time conversations more inclusive and accessible. This project introduces a communication service designed specifically for deaf and mute people, enabling smooth and natural interaction with anyone through live translation and video-based communication features.

The proposed application allows users to enter the system by selecting whether they are impaired or not. During a video call, if one user is hearing or speech impaired and the other is normal, the application intelligently adapts the communication mode. The normal user can speak naturally, and the impaired person receives the conversation as live text captions or sign language animations. Similarly, when the impaired user responds using sign language, the system detects their gestures through the camera, converts them into text, and finally into voice output for the normal user. This creates a two-way translation that breaks communication barriers effectively.

The system also provides a live translation feature for real-world interactions. By using the camera, a user can capture a person performing sign language gestures, and the app will translate those signs into text or spoken words instantly. This feature extends beyond video calls and can be used in daily life situations such as workplaces, hospitals, or public spaces, ensuring independence and confidence for hearing or speech-impaired individuals.

1.1 Problem Statement

People with hearing and speech impairments struggle to communicate with others in real-time scenarios, especially with those unfamiliar with sign language. Existing communication meth-

ods rely heavily on interpreters, typing, or pre-recorded phrases, which are inefficient and hinder natural interaction. The lack of real-time translation tools and accessible communication services limits their participation in social, educational, and professional environments.

1.2 Objective

1. To develop a communication platform that enables seamless real-time conversation between impaired and normal users using AI-based translation and sign language recognition.

2. To provide live captions and sign language visuals for spoken communication, allowing impaired users to understand conversations without external assistance.

3. To translate sign language gestures into text and speech, enabling normal users to easily understand and respond during a conversation.

1.3 Scope

1. Users can choose their role (impaired or normal) upon entering the app to personalize communication features and accessibility tools.

2. The system supports real-time video calling where voice is automatically converted to text or sign animations, and sign gestures are translated into voice output.

3. The app includes a live translation mode where users can point their camera toward someone using sign language, and the system translates it into speech instantly.

4. The application can be used across multiple domains such as education, healthcare, workplaces, and public service centers to facilitate communication without interpreters.

5. The project leverages AI, speech recognition, and computer vision technologies to create an inclusive platform that ensures accessibility, independence, and equal communication opportunities for all users.

Chapter 2

LITERATURE SURVEY

2.1 STSLR: Sign Language Recognition With Self-Learning Fusion Model

This paper introduces a novel self-learning fusion approach for SLR, termed STSLR, that bridges video and sensor modalities through simulated wearable-sensor features generated directly from video data. The system enhances recognition performance in cases where only video is available during testing. The authors also propose a new Vietnamese Sign Language (VSL) dataset comprising synchronized video and smartwatch inertial data with 50 sign classes and nearly 8000 samples, enabling a deeper exploration of multimodal fusion techniques.

The motivation behind this study stems from the limitations of traditional vision-based and sensor-based SLR systems. Vision-based methods are prone to performance degradation under poor lighting, occlusion, or varying camera angles, while sensor-based systems require wearable hardware and are costly to scale. To overcome these issues, STSLR introduces a learning framework that generates sensor-like features from video and combines them with visual embeddings in a unified recognition pipeline.[1]

2.1.1 Methodology of STSLR Model

The STSLR framework consists of two main stages: sensor feature simulation and multimodal fusion. In the first stage, two networks-an Inception-1D model for real sensor data and an Inception-3D model for video-are trained together using a combination of cross-entropy and Huber loss functions. The Inception-3D model learns to generate simulated sensor embeddings that closely resemble those extracted from real inertial data. In the second stage, these generated embeddings are fused with video features through a feature-level fusion process and fed into a Fully Connected Network (FCN) for final classification.

The proposed model was trained and evaluated on the newly collected Vietnamese Sign Language (VSL) dataset, which includes 15 participants performing 50 distinct signs. Each sample consists of RGB video recorded at 30 fps and corresponding smartwatch IMU data (3-axis accelerometer and gyroscope). The dataset underwent a session-wise split (80:20) to ensure no overlap of participants between training and testing. Performance was further validated on public datasets such as UTD-MHAD and Berkeley MHAD for cross-domain comparison.

The evaluation results demonstrate that STSLR achieves superior accuracy compared to state-of-the-art baselines like Swin-B and I3D. When tested using only video input, STSLR reaches an F1-score of 87.1%, outperforming traditional vision models. When real sensors are available, the combined performance slightly improves to 89.4%, validating the efficiency of simulated sensor generation. The use of the Huber loss function provided better stability and robustness against outliers compared to MSE and MAE.

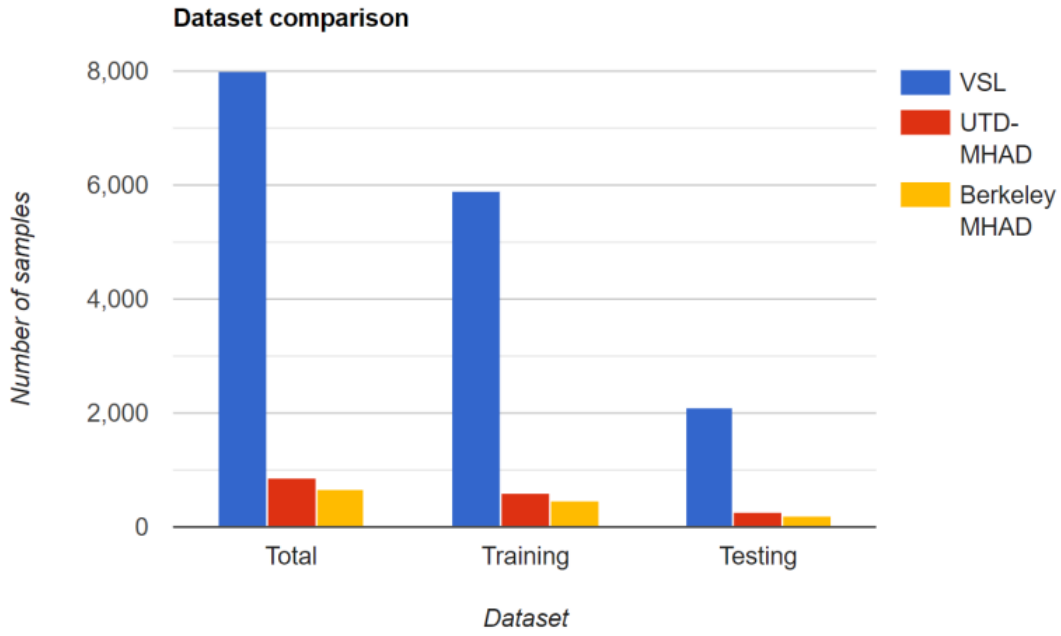

Figure 2.1: Comparison of Vietnamese Sign Language (VSL) dataset with existing multimodal datasets such as UTD-MHAD and Berkeley MHAD.

2.1.2 Model Architecture

The architecture of the proposed model is illustrated below. The system first extracts features from both real sensors and video streams, learns to generate simulated sensor embeddings from video, and finally fuses these embeddings for classification.

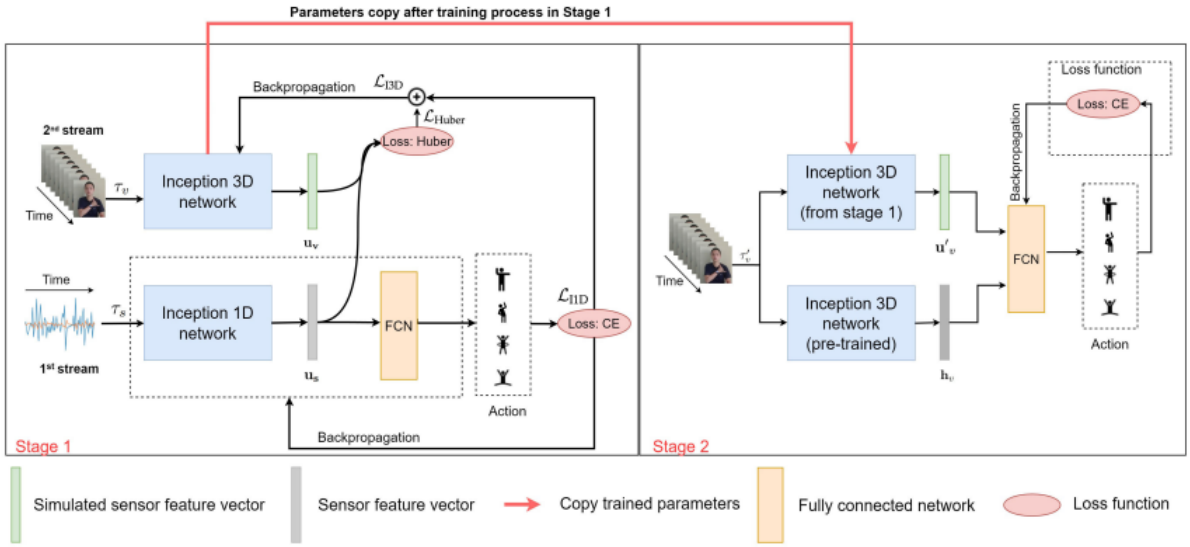

Figure 2.2: Architecture of the proposed STSLR model.

2.1.3 Advantages

1. **Practical Implementation:** STSLR enables real-time sign recognition using only video input at inference, reducing the need for wearable devices.
2. **Robust Fusion:** The use of Huber loss and feature-level fusion yields more stable and accurate results across multiple datasets.
3. **Scalable Dataset Contribution:** The proposed VSL dataset fills a crucial gap for multimodal sign language research and is expandable for future studies.

2.1.4 Disadvantages

1. **Single User Limitation:** The model assumes a single performer per frame, which limits performance in multi-person or complex environments.
2. **Limited Finger Detail:** The reliance on wrist-mounted IMUs restricts fine-grained recognition of intricate finger movements.
3. **Reduced Generalization:** Performance may degrade in uncontrolled lighting or occluded conditions compared to controlled lab environments.

2.2 SAW: Semantic-Aware WebRTC Transmission for Real-Time Communication Systems

This paper presents a novel approach named SAW that enhances real-time video communication systems by integrating semantic awareness into WebRTC transmission. The objective of SAW is to improve bandwidth efficiency, transmission stability, and perceptual video quality by dynamically prioritizing semantically important regions during video encoding and streaming. The authors propose a complete semantic-aware pipeline that operates within standard WebRTC protocols, ensuring compatibility and deployability in existing real-time applications such as conferencing and remote collaboration.[2]

The motivation arises from the limitations of traditional WebRTC, which treats all parts of a video frame equally, leading to unnecessary bitrate consumption for unimportant regions. With the rapid rise of AI-driven vision systems, it becomes increasingly important to allocate network resources intelligently based on content importance. SAW leverages deep learning-based semantic segmentation to identify regions of interest and applies adaptive encoding and selective packet transmission, thereby improving Quality of Experience (QoE) under limited bandwidth conditions.

2.2.1 Methodology of SAW Transmission Model

The proposed SAW framework integrates three main modules within the WebRTC stack: the **Semantic Analysis Module**, the **Encoder and Transmission Control Module**, and the **Semantic-Aware Feedback Mechanism**. The semantic analysis module performs real-time segmentation of each video frame to identify meaningful regions such as faces, objects, or active speakers. The encoder and transmission module then adaptively compresses these important regions with higher quality while applying stronger compression to background areas. Meanwhile, the feedback mechanism continuously monitors network conditions to dynamically balance bitrate allocation and maintain low latency.

The system was implemented and evaluated using the open-source WebRTC framework. A Deep Neural Network (DNN) model based on U-Net architecture was trained for semantic segmentation to run alongside real-time video encoding. The integration was optimized to run efficiently on edge devices without significant latency overhead. Extensive experiments were conducted under varying bandwidth and packet loss conditions to test adaptability and robustness.

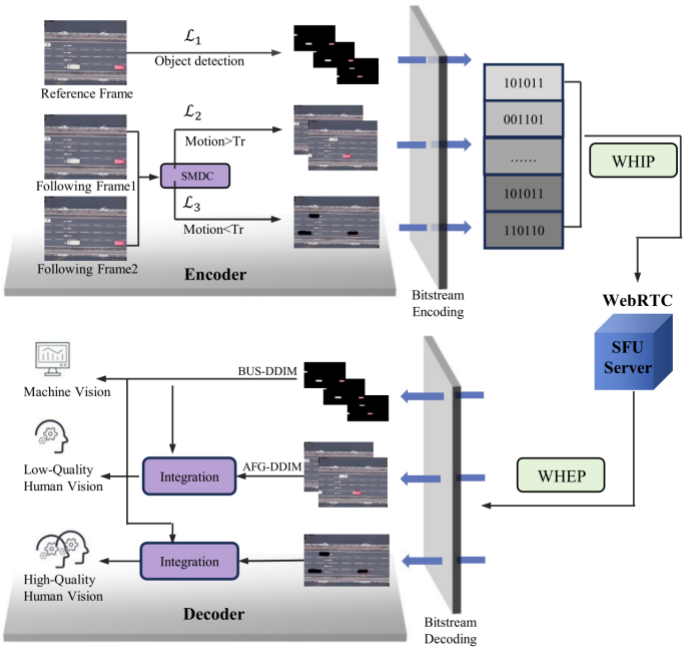

Figure 2.3: Architecture of the proposed SAW framework

Experimental evaluations demonstrated that SAW achieved up to 32% bandwidth savings while maintaining comparable or higher perceptual quality (measured by VMAF and SSIM scores) compared to baseline WebRTC. Under network congestion, SAW provided smoother frame delivery and reduced freezing rates. Subjective user studies also confirmed noticeable visual improvement in key semantic regions such as faces and gestures.

2.2.2 Advantages

1. **Bandwidth Efficiency:** Focuses transmission on important visual regions, reducing data usage.
2. **Seamless Integration:** Works with existing WebRTC systems with minimal modification.
3. **Adaptive Control:** Adjusts quality dynamically based on network conditions.

2.2.3 Disadvantages

1. **Processing Load:** Real-time segmentation increases computational demand.
2. **Model Reliance:** Accuracy depends on proper semantic region detection.
3. **Visual-Only Focus:** Does not yet utilize audio or contextual semantics.

2.3 Understanding and Taming the Inflated Latency in Mobile Cloud Rendering

This paper, addresses the critical issue of high latency in mobile cloud rendering systems that enable real-time 3D graphics on mobile devices. Cloud rendering allows computationally intensive rendering tasks to be offloaded to powerful servers, but maintaining low *Motion-to-Photon (MTP)* latency remains a key challenge. The authors identify that the primary cause of inflated latency lies in the *Receive-to-Composition (R2C)* stage, which is mainly affected by ineffective jitter buffer management on the client side.[3]

To mitigate this issue, the authors propose a lightweight, client-side optimization framework named JitBright. It dynamically adjusts jitter buffer levels using an adaptive gain control mechanism and proactively requests keyframes to minimize both active and passive waiting times. JitBright intelligently balances low latency and playback smoothness by adapting to different device grades and network conditions (WiFi, 4G, and 5G). The system was evaluated using more than 591,000 real-world sessions, achieving up to 87.5% reduction in R2C latency and up to 27% improvement in sessions meeting strict latency thresholds.

2.3.1 Methodology of the JitBright System

The paper begins with a detailed measurement study that decomposes MTP latency into three main components: (i) Rendering Engine (RE) latency, (ii) Network Transmission (NT) latency, and (iii) Receive-to-Composition (R2C) latency. Measurements from large-scale real-world sessions reveal that R2C latency dominates the overall MTP latency. This latency arises primarily from buffering delays in the jitter buffer, where video frames experience either active waiting (held for playback synchronization) or passive waiting (delayed by missing reference frames).

JitBright introduces three major modules - the *Latency Diagnostic*, *Jitter Delay Manager*, and *Keyframe Requester*. The latency diagnostic module analyzes network type and device performance to tune parameters in real time. The jitter delay manager employs an *adaptive gain mechanism* to modify buffer levels based on playback smoothness probability, while the keyframe requester proactively mitigates decoding dependencies by predicting when to request keyframes. These modules operate collaboratively to minimize both active and passive waiting, improving overall playback responsiveness.

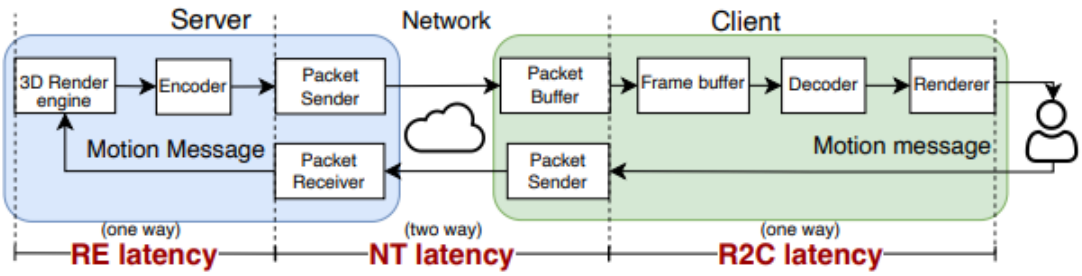

Figure 2.4: Real-time cloud rendering architecture and latency decomposition.

The experimental results demonstrate that JitBright achieves consistent latency reduction across diverse environments. On WiFi, 4G, and 5G networks, median R2C latency dropped by 82.4%, 86.5%, and 87.5%, respectively. Moreover, JitBright reduced the video freeze rate from 2.4–2.8% to below 1%, without compromising playback smoothness. The adaptive gain control enables the system to automatically fine-tune its parameters to the user's current device and network conditions, ensuring scalability and real-world applicability.

2.3.2 Advantages

1. **Significant Latency Reduction:** Achieves up to 87.5% reduction in R2C latency and improved motion-to-photon responsiveness.
2. **Adaptive Optimization:** Adjusts automatically for different devices and network conditions.
3. **Improved Playback Smoothness:** Reduces video freeze rates to below 1% while maintaining quality.

2.3.3 Disadvantages

1. **Client-Side Complexity:** Requires modification of WebRTC's jitter buffer module and integration with rendering clients.
2. **Limited by Network Quality:** Performance may degrade under severe packet loss or unstable connections.
3. **Computational Overhead:** Although lightweight, continuous monitoring adds minor processing cost to mobile devices.

2.4 Collaborative Multilingual Continuous Sign Language Recognition: A Unified Framework

Continuous Sign Language Recognition (CSLR) systems have traditionally been developed for individual languages, limiting their scalability and cross-lingual generalization. This paper presents a unified multilingual framework that collaboratively trains on multiple sign languages, leveraging shared visual-linguistic patterns to enhance representation learning. The proposed model integrates a shared visual encoder with both language-specific and universal sequential modules, enabling the joint modeling of common and language-dependent features. Evaluations on the RWTH-PHOENIX-Weather (German Sign Language), CSL (Chinese Sign Language), and GSL-SD (Greek Sign Language) datasets demonstrate that the framework achieves superior performance over single-language models, improving recognition accuracy and robustness, particularly in low-resource scenarios.[4]

2.4.1 Methodology

The framework is designed following a comparative study of baseline CSLR architectures, adopting a hybrid design that balances shared and language-specific learning. A shared CNN-TCN visual encoder extracts spatiotemporal features from sign video sequences, capturing motion and handshape patterns common across sign languages. These representations are then processed by Bidirectional LSTM (BLSTM) modules-one shared across all languages and one specific to each language-to model temporal dependencies.

Language embeddings are used to initialize the shared BLSTM, encoding linguistic identity and enabling adaptive feature interpretation. A novel *max-probability decoding* algorithm is introduced to align video frames with gloss sequences, improving temporal correspondence and facilitating iterative refinement of the encoder through Connectionist Temporal Classification (CTC) loss. This collaborative training strategy encourages the transfer of visual-linguistic knowledge across languages while preserving language-specific nuances.

2.4.2 Architecture

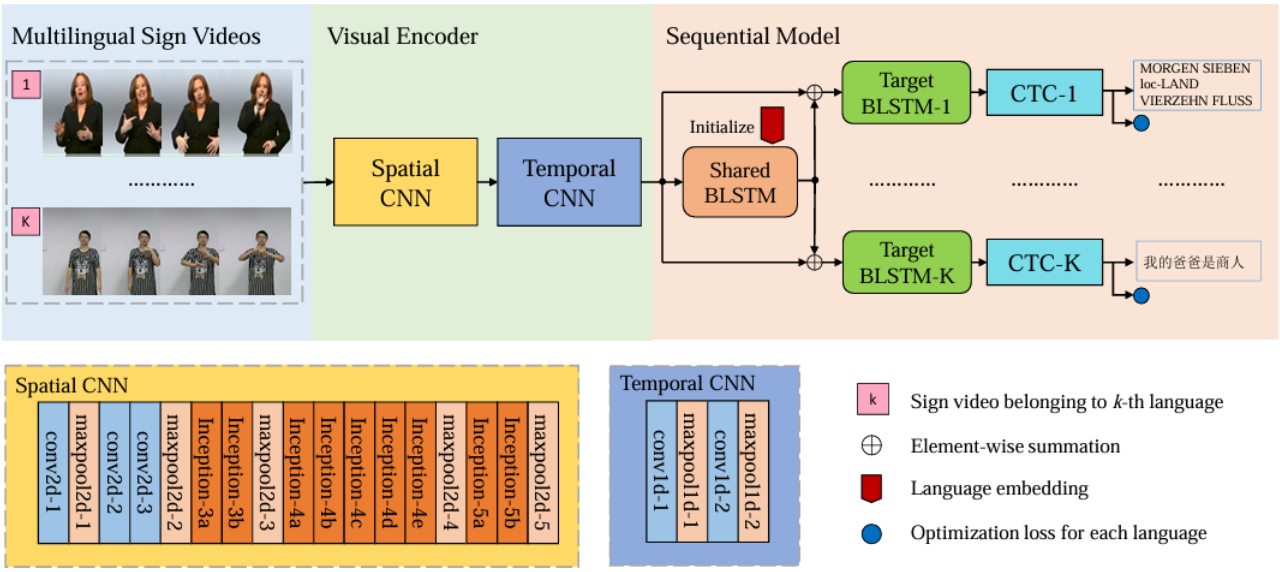

Figure 2.5: Multilingual CSLR Architecture

2.4.3 Advantages

1. **Collaborative Learning:** Leverages shared visual patterns across languages, enhancing recognition accuracy and cross-lingual generalization.
2. **Low-Resource Adaptability:** Multilingual training increases data diversity, reducing overfitting and improving performance for low-resource languages.
3. **Modular and Scalable Design:** Separate sequential modules per language allow for fine-grained linguistic modeling while supporting easy integration of new languages.

2.4.4 Disadvantages

1. **Higher Model Complexity:** Incorporating multiple sequential modules and language embeddings increases computational and architectural complexity.
2. **Language-Specific Optimization:** Despite shared representations, each language still requires fine-tuning to capture unique grammatical and lexical structures.
3. **Data Imbalance Sensitivity:** Unequal dataset sizes may bias the shared encoder toward high-resource languages, affecting fairness across tasks.

2.5 MASA: Motion-aware Masked Autoencoder with Semantic Alignment for Sign Language Recognition

The increasing demand for robust SLR systems has highlighted limitations in existing pre-training approaches, particularly their neglect of explicit motion cues and global semantic context. To address these challenges, we propose MASA, a Motion-aware Masked Autoencoder with Semantic Alignment. MASA introduces a defense-in-depth inspired architecture for SLR, comprising two core modules: a motion-aware Masked Autoencoder (MA) and a momentum Semantic Alignment (SA) module. MA focuses on reconstructing motion residuals from masked frames to capture dynamic pose variations, while SA aligns global semantic features across augmented samples to enhance instance-level representation. MASA achieves state-of-the-art performance across four public benchmarks and demonstrates superior efficiency in terms of energy, communication, and computational cost. The framework is implemented using PyTorch and validated on NVIDIA RTX 3090 GPUs.[5]

2.5.1 Proposed SLR Framework

Sign pose sequences, motion residuals, confidence scores, and semantic embeddings are the key elements in the MASA framework. Technologies involved include Graph Convolutional Networks (GCNs), Transformer architectures, and contrastive learning mechanisms. MASA integrates motion-aware masking and semantic consistency learning to enhance representation quality.

The motion-aware masked strategy selects high-confidence, high-motion frames for masking, avoiding redundant or low-quality data. The MA module reconstructs motion residuals using a Transformer-based encoder-decoder pipeline. The SA module generates augmented samples, projects them into a shared latent space, and aligns their embeddings using a contrastive loss. Momentum updates ensure stable training dynamics.

MASA does not rely on RGB-based features or network-layer security. Instead, it leverages pose-based data and self-supervised learning to build robust, generalizable models. Semantic alignment ensures that the model captures both local motion and global sign meaning, improving recognition accuracy across diverse signers and environments.

2.5.2 Model Architecture

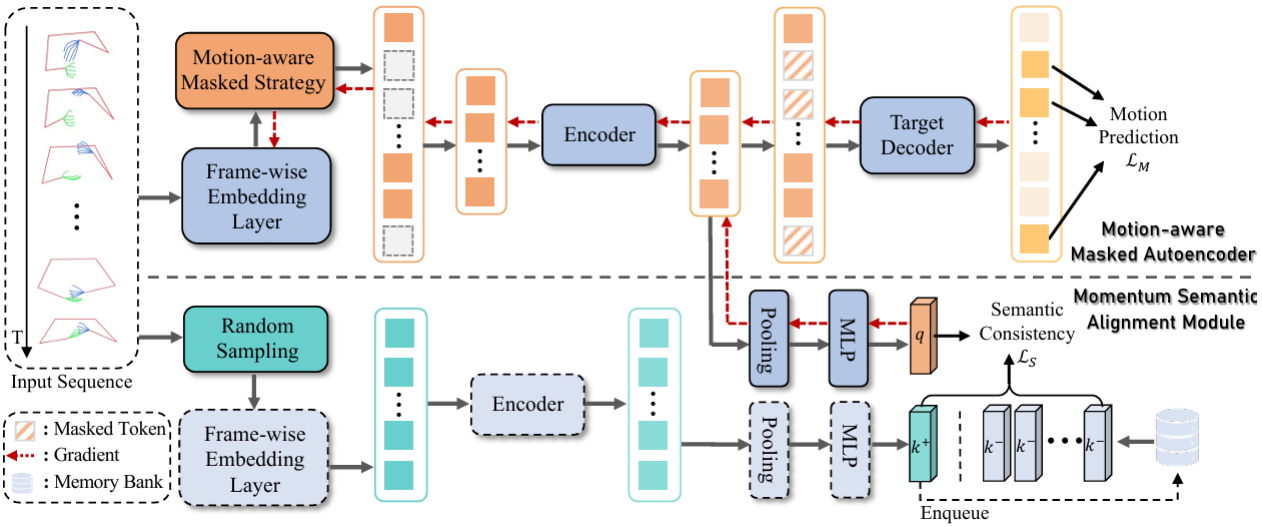

Figure 2.6: MASA Framework: Motion-aware Masked Autoencoder and Semantic Alignment Module

2.5.3 Advantages

1. **Dual-Module Design:** MASA's combination of motion-aware masking and semantic alignment enables comprehensive learning of both dynamic and contextual features in sign pose sequences.
2. **Robust Against Pose Noise:** By incorporating confidence thresholds and motion residuals, MASA effectively filters unreliable pose data and focuses on meaningful motion cues.
3. **Verified Performance:** MASA achieves state-of-the-art results on WLASL, MSASL, NMFs-CSL, and SLR500 benchmarks, demonstrating its effectiveness across varied datasets.

2.5.4 Disadvantages

1. **Computational Overhead:** The dual-module architecture and Transformer-based design may introduce higher computational requirements during training.
2. **Complexity in Augmentation Strategy:** Designing effective temporal augmentations and maintaining semantic consistency across samples can be challenging and may require careful tuning.

2.6 Word Level Sign Language Recognition via Handcrafted Features

The proposed system explores enhancing SLR by utilizing handcrafted features for word-level classification. Unlike deep learning approaches that rely on large feature spaces and high computational resources, handcrafted features offer advantages like low complexity and suitability for real-time applications. Challenges addressed include occlusion, non-manual feature integration, and efficient recognition. The proposed methodology is validated through implementing a recognition pipeline using Bidirectional Long Short-Term Memory (BiLSTM) and Transformer models, demonstrating high accuracy on the LIBRAS dataset. Experiments affirm the practical feasibility and effectiveness of this approach, contributing to the advancement of lightweight SLR systems.[6]

2.6.1 Methodology

This study focuses on enhancing word-level Sign Language Recognition using handcrafted spatiotemporal features derived from manual and non-manual gestures. The goal is to reduce model complexity while retaining discriminative power. The proposed methodology involves region-of-interest detection using MediaPipe and YOLOv5 for hands, and OpenFace for head pose and gaze estimation. Feature extraction includes spatial coordinates, Euclidean distances between key points, and motion-based metrics. Data augmentation is performed via joint rotation to simulate signer variability, and Principal Component Analysis (PCA) is applied for dimensionality reduction. Evaluation includes implementing BiLSTM and Transformer models for classification. The methodology is presented in the context of recognizing isolated sign words from continuous video input, with emphasis on non-manual features like gaze and head orientation. The paper concludes with a summary of results and implications.

2.6.2 Architecture

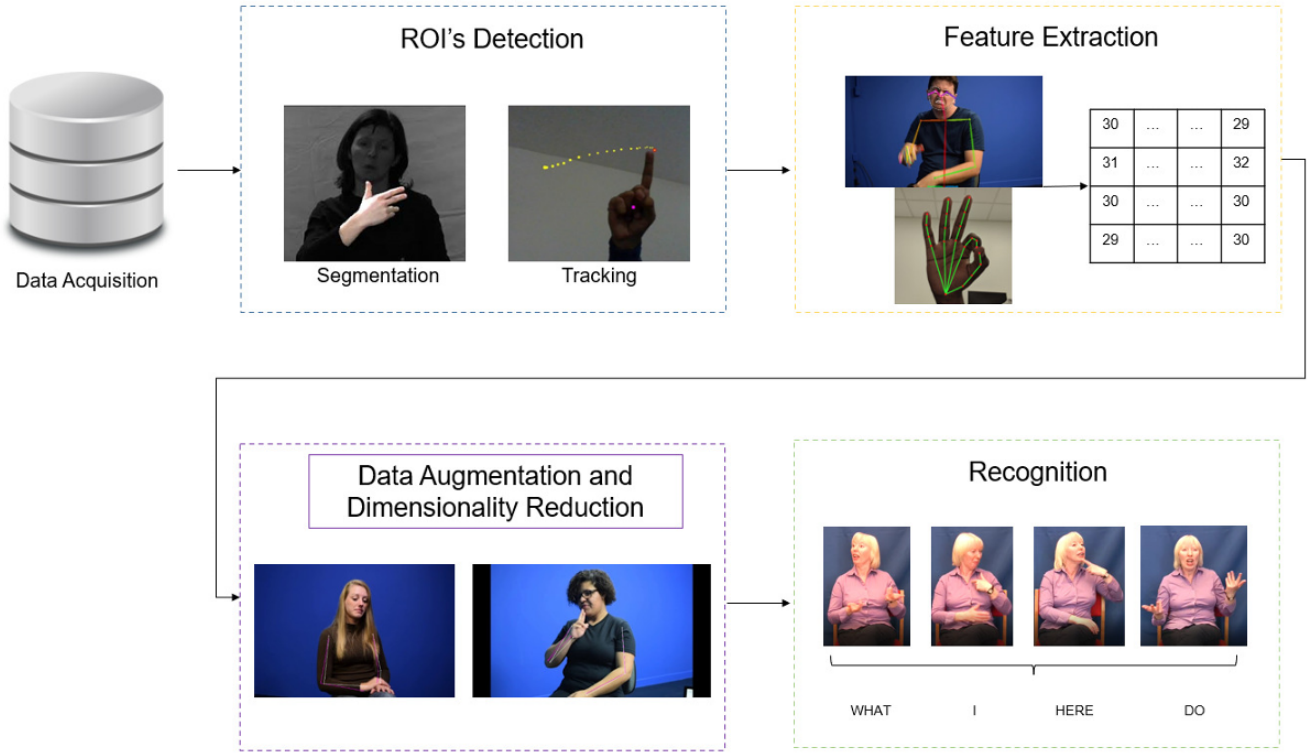

Figure 2.7: Architecture Diagram

2.6.3 Advantages

1. **Lightweight Feature Space:** Utilizing handcrafted features enables efficient processing and reduces the need for high-end hardware, making the system suitable for mobile and real-time applications.

2. **Non-Manual Feature Integration:** Incorporating gaze direction and head pose enhances recognition accuracy and addresses gaps in prior work that overlooked these expressive cues.

2.6.4 Disadvantages

1. **Occlusion Sensitivity:** Hand pose estimation may fail under frequent occlusions, requiring robust detection strategies and fallback mechanisms.

2. **Annotation Limitations:** Dataset annotations may be incomplete or inconsistent, affecting training quality and generalization across diverse signers and contexts.

2.7 Prior-Aware Cross Modality Augmentation Learning for Continuous Sign Language Recognition

Continuous Sign Language Recognition (CSLR) aims to translate continuous sign videos into corresponding text sentences. Traditional deep-learning-based CSLR systems often rely on the Connectionist Temporal Classification (CTC) loss for optimization, which maximizes the posterior probability of sequence alignment. However, there exists a performance gap between CTC loss and the Word Error Rate (WER)-the main evaluation metric for CSLR-since the sequence with the highest CTC probability is not always the one with the lowest WER. To address this issue, this paper introduces a Prior-Aware Cross Modality Augmentation Learning (PCMA) framework that enhances CSLR by generating high-quality pseudo video-text pairs guided by linguistic and visual priors. These pseudo samples act as challenging examples, helping the model learn fine-grained distinctions and reducing the mismatch between CTC optimization and WER evaluation.[7]

2.7.1 Methods and Materials

The proposed PCMA framework performs cross-modality editing-substitution, insertion, and deletion-on paired real video-text data to create pseudo pairs. To ensure realism, the method integrates two forms of prior knowledge:

Textual Grammar Prior: Built using a BERT-like masked language model that captures the joint text distribution and maintains syntactic correctness during editing.

Visual Pose Transition Prior: Based on upper-body 2D pose estimation using MMPose, which maintains temporal and spatial consistency in video transitions.

The framework consists of three main components:

A visual encoder (2D-CNN + Temporal Convolution Network) to extract spatio-temporal features from videos.

A BLSTM-based sequential model trained with CTC loss to align visual features with gloss sequences.

A text encoder (two-layer BLSTM) that maps textual input into the same latent semantic space as visual data.

Training uses both real and pseudo pairs and optimizes the model with three loss terms:

Alignment Loss (CTC): Ensures correspondence between video and text sequences.

Semantic Correspondence Loss: Aligns features from both modalities within a unified latent space using dynamic time warping (DTW).

Real–Pseudo Discriminative Loss: Encourages the model to distinguish real from pseudo pairs using triplet loss, enhancing fine-grained discriminability.

The overall objective function combines these loss terms using a weighted factor , achieving an optimal trade-off between alignment and discriminative learning.

2.7.2 Proposed System Model

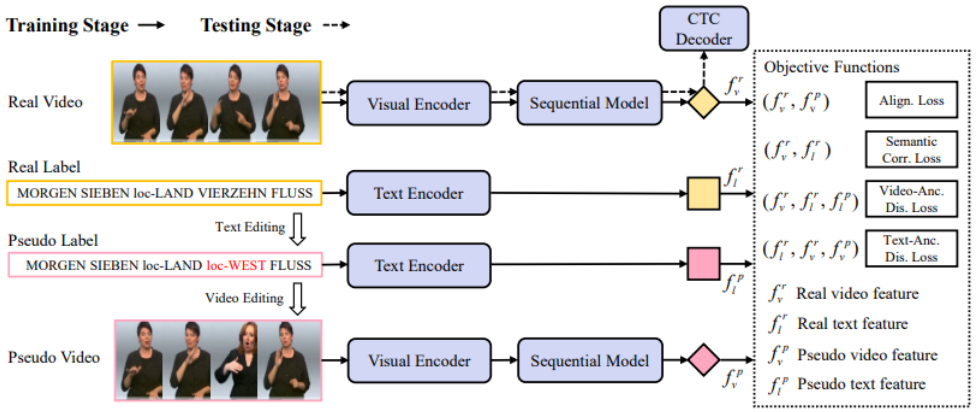

Figure 2.8: Overview of the Prior-Aware Cross Modality Augmentation Learning Framework

2.7.3 Advantages

1. **Reduced Optimization Gap:** The method directly targets the inconsistency between CTC loss and WER, leading to more accurate predictions.
2. **High-Quality Data Augmentation:** Incorporating linguistic and visual priors ensures realistic pseudo samples that strengthen model generalization.
3. **Enhanced Robustness:** The multi-loss optimization and discriminative training improve performance across various benchmark datasets, including RWTH-Phoenix and CSL.
4. **State-of-the-Art Results:** The model achieves significant WER reductions-down to 20.0

2.7.4 Disadvantages

1. **Computational Overhead:** The use of multiple encoders and losses increases training complexity and computational requirements.
2. **Data Dependency:** The model relies on large-scale annotated sign language datasets and precise video-text alignment for optimal results.
3. **Hardware Limitations:** Pose extraction and visual editing demand significant computational power and may not be practical in real-time applications.

2.8 Semantic-Preserved Communication System for Highly Efficient Speech Transmission

A novel **semantic-preserved communication system** is proposed for highly efficient speech transmission that integrates the principles of semantic communication and neural networks. Unlike traditional speech transmission methods that rely heavily on bit-level signal accuracy, the proposed system focuses on preserving the *meaning* (semantics) of speech rather than its raw waveform. This allows for improved robustness against channel noise and a significant reduction in transmission bandwidth.

The system employs a deep learning-based architecture that extracts, transmits, and reconstructs semantic features rather than full audio signals. This approach leads to a higher level of communication efficiency, as only essential semantic information is shared over the channel, maintaining speech intelligibility and speaker identity even under noisy environments.[8]

2.8.1 Proposed System Model

The proposed semantic-preserved communication system consists of three key modules: the **Semantic Encoder**, the **Channel Encoder-Decoder**, and the **Semantic Decoder**.

1. **Semantic Encoder:** The encoder extracts meaningful semantic information from the speech input using a deep neural network trained to capture linguistic and acoustic features simultaneously. This ensures that only the most relevant speech representations are transmitted.

2. **Channel Encoder-Decoder:** The extracted semantic embeddings are transmitted through a noisy channel, modeled by additive white Gaussian noise (AWGN). A lightweight channel encoder compresses and protects the transmitted semantic vectors. The decoder then reconstructs these vectors with minimal distortion using learned denoising techniques.

3. **Semantic Decoder:** The receiver-side decoder reconstructs the speech signal from the transmitted semantic features. This reconstruction focuses on preserving intelligibility and tone rather than the precise waveform, achieving communication efficiency even under degraded channel conditions.

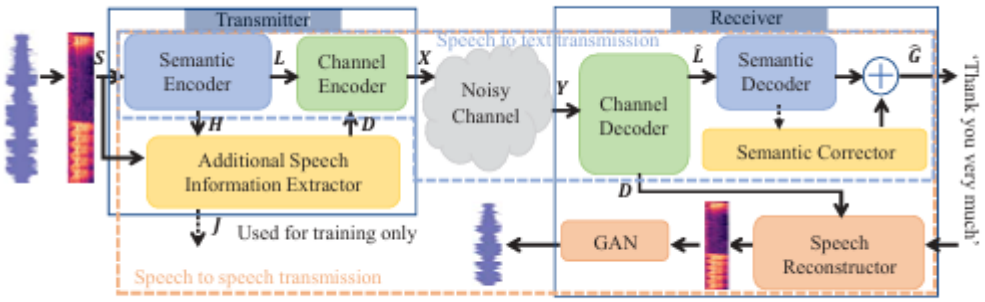

Figure 2.9: The system model for speech transmission.

2.8.2 System Architecture and Methodology

The proposed architecture adopts an end-to-end deep learning design trained with a semantic loss function. Instead of using traditional Bit Error Rate (BER) as the optimization objective, the system employs a semantic similarity loss, which measures how well the decoded message preserves meaning relative to the original.

The training process involves joint optimization of both semantic and channel modules. The encoder and decoder are jointly trained to achieve robustness against channel variations, and an attention mechanism is introduced to highlight semantically important speech segments. Additionally, the system supports variable transmission rates, adapting to different channel capacities dynamically.

To evaluate performance, the authors conducted experiments using speech datasets transmitted over noisy channels. The results demonstrated that the semantic-preserved system outperforms conventional waveform-based communication methods in both intelligibility and Signal-to-Noise Ratio (SNR) performance, while using less bandwidth.

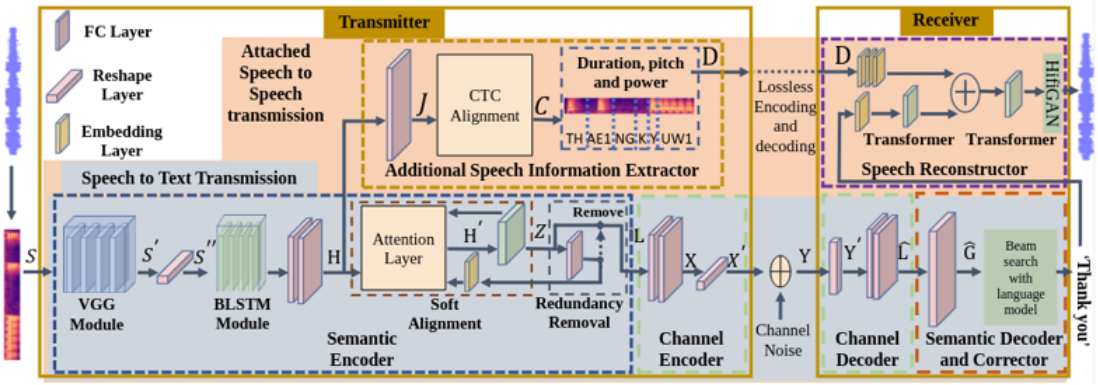

Figure 2.10: The proposed system architecture for the speech transmission semantic communication system.

2.8.3 Advantages

1. **High Efficiency:** By transmitting semantic features instead of raw waveforms, the system significantly reduces bandwidth usage while maintaining speech clarity.
2. **Robustness to Noise:** The semantic-level representation is resilient to channel noise and distortions, outperforming traditional speech codecs in adverse conditions.
3. **End-to-End Optimization:** Joint training of the encoder, channel, and decoder modules ensures consistent performance across various communication environments.
4. **Preserved Intelligibility:** The system effectively maintains the meaning of speech messages, ensuring clear understanding even when signal quality is low.

2.8.4 Disadvantages

1. **High Computational Complexity:** The training and real-time implementation of deep semantic models require significant computational resources and specialized hardware.
2. **Dependency on Training Data:** The system's performance depends heavily on the quality and diversity of the training dataset, which may limit generalization to unseen speakers or languages.
3. **Potential Latency:** The neural processing pipeline may introduce additional latency compared to traditional digital modulation systems.
4. **Limited Interpretability:** Since the system learns semantic compression through deep learning, interpreting how meaning is represented internally remains a challenging task.

2.9 Scaling Up Multimodal Pre-training for Sign Language Understanding

A large-scale multimodal pre-training framework for Sign Language Understanding (SLU) is introduced to enhance model generalization and performance across various SLU tasks. Traditional pre-training approaches often rely on limited datasets or focus solely on visual cues, which limits their ability to learn rich semantic representations. To address these issues, a multimodal Sign Language Pre-training (SLP) method is proposed, integrating visual and textual information using a newly curated large-scale dataset named SL-1.5M.

The SL-1.5M dataset, consisting of approximately 1.5 million sign-text pairs, is compiled from multiple existing sign language datasets to overcome data scarcity. The framework employs two major pretext tasks: sign-text contrastive learning and masked pose modeling. This combination enables the model to capture both contextual and semantic relationships between sign gestures and their corresponding textual representations. Experiments on various SLU tasks such as isolated and continuous sign recognition, sign translation, and sign retrieval demonstrate the proposed framework's effectiveness, achieving new state-of-the-art results on several benchmarks.[9]

2.9.1 Proposed Framework and Model Architecture

The proposed model architecture consists of three main components: a Sign Pose Encoder, a Multilingual Text Encoder, and a Sign Pose Decoder. The design of these modules ensures the extraction of meaningful visual features and semantic alignment with linguistic information.

The Sign Pose Encoder uses a dual-branch structure to separately process manual gestures (hand and arm movements) and non-manual cues (facial and mouth expressions). These extracted features are combined to represent complete sign semantics. The Text Encoder, based on the pre-trained multilingual MBART model, captures language semantics across English, German, and Chinese. To maintain language integrity, its parameters remain frozen during pre-training.

The Sign Pose Decoder reconstructs the masked portions of sign sequences using the learned contextual cues, ensuring that the model develops a comprehensive understanding of gesture continuity. The training framework aligns visual and textual modalities through a fine-grained similarity module, effectively narrowing the semantic gap between visual and linguistic representations.

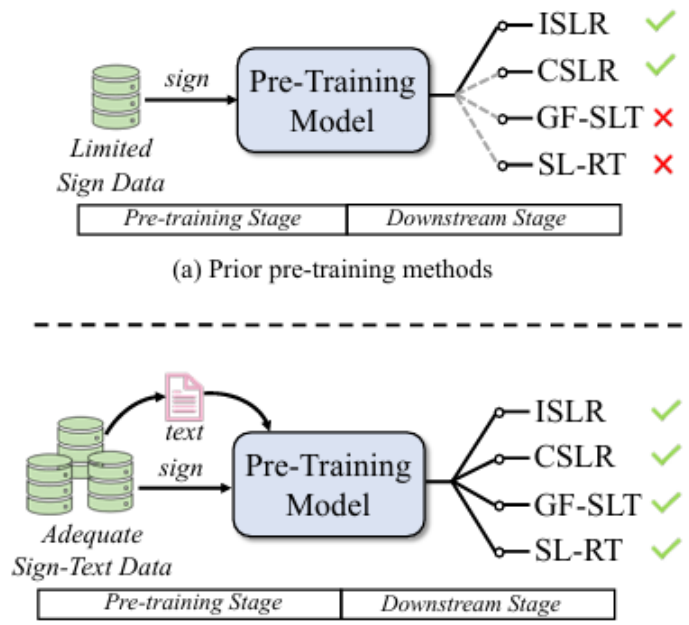

Figure 2.11: Overview of (a) prior pre-training methods [1]–[4] and (b) paper proposed method.

2.9.2 Dataset Description: SL-1.5M

The SL-1.5M dataset is a large-scale collection of paired sign pose and text data derived from multiple public resources, including WLASL, MSASL, CSL-Daily, How2-Sign, Phoenix14, and BOBSL. It includes videos from various sign languages such as American (ASL), British (BSL), Chinese (CSL), and German (GSL), resulting in high linguistic diversity.

Pose keypoints are extracted using MMPose and InterWild estimators to represent detailed hand, face, and upper-body motions, totaling 79 2D keypoints per frame. The dataset covers approximately 1,780 hours of signing videos and includes over 600 individual signers. To balance language diversity, additional hand pose data are generated for underrepresented languages, improving model robustness during pre-training.

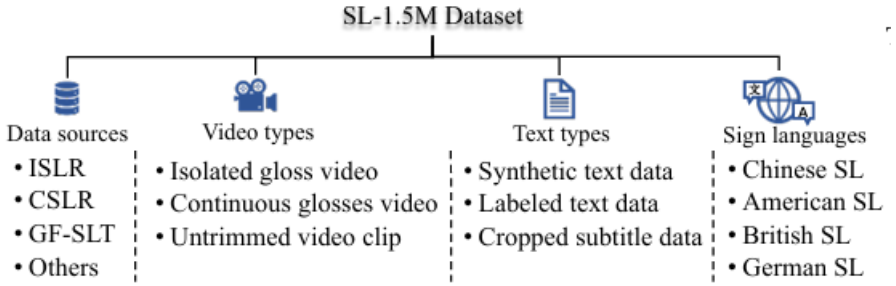

Figure 2.12: The key composition of SL-1.5M dataset.

2.9.3 Pre-training and Fine-tuning Methodology

During pre-training, two learning strategies are applied:

1. **Masked Pose Modeling:** Random segments of the pose sequence are masked, and the model learns to reconstruct missing joints using contextual information.
2. **Sign-Text Contrastive Learning:** Sign and text representations are aligned in a shared latent space, ensuring that related pairs are brought closer together while unrelated pairs are pushed apart.

After pre-training, the model is fine-tuned for several downstream SLU tasks:

1. **ISLR (Isolated Sign Language Recognition):** Uses an Encoder + MLP classifier.
2. **CSLR (Continuous Sign Language Recognition):** Incorporates BiLSTM and CTC loss for temporal sequence alignment.
3. **GF-SLT (Gloss-Free Sign Language Translation):** Employs a transformer-based text decoder for end-to-end translation.
4. **SL-RT (Sign Language Retrieval):** Uses contrastive learning to match sign videos with corresponding text queries.

2.9.4 Advantages

1. **Multimodal Learning:** Combines visual and textual inputs for improved semantic understanding.
2. **Extensive Dataset:** SL-1.5M ensures diverse, generalizable model training.
3. **Task Flexibility:** Easily adapts to multiple SLU applications.
4. **Comprehensive Features:** Dual-branch extraction captures full sign semantics.

2.9.5 Disadvantages

1. **High Computation:** Requires substantial processing power and GPUs.
2. **Data Imbalance:** Some languages and signs remain underrepresented.
3. **Heavy Preprocessing:** Pose and data alignment are resource-intensive.
4. **Limited Testing:** Real-world performance not yet fully validated.

2.10 End-to-End Multi-Modal Speech Recognition on an Air and Bone Conducted Speech Corpus

Automatic Speech Recognition (ASR) has achieved remarkable success with deep learning architectures such as DNN-HMM and end-to-end transformer-based models. However, traditional ASR systems rely solely on Air-Conducted (AC) speech, making them highly susceptible to environmental noise and low Signal-to-Noise Ratios (SNR). This study introduces a novel multi-modal ASR framework that integrates AC and Bone-Conducted (BC) speech to enhance robustness in noisy conditions. The BC microphone, being relatively insensitive to ambient noise, complements AC input by providing clearer speech cues in adverse environments. The authors present a large-scale synchronized Air- and Bone-Conducted Speech Corpus (ABCS) in Mandarin, recorded from 100 speakers using a dual-channel headset-marking a significant advancement in bone-conducted speech datasets.[10]

2.10.1 Methods and Materials

The proposed approach combines AC and BC speech using a multi-modal conformer-based ASR architecture enhanced with a Multi-Modal Transducer (MMT). This transducer dynamically fuses semantic embeddings from both modalities through adaptive weight allocation, leveraging conformer encoders and transformer-based truncated decoders to extract high-level contextual features.

The ABCS corpus contains 47,182 synchronized AC and BC utterances, totaling 42 hours of valid data from balanced gender participants aged 20–35. Data were recorded in an anechoic chamber following ISO 3745 standards. Spectrogram and power spectral density analyses revealed that BC speech retains low-frequency components (below 1 kHz) effectively but suffers from high-frequency attenuation and self-noise. Mutual information analysis demonstrated that BC speech contains semantic information comparable to noisy AC speech at 5 dB SNR, confirming its potential for robust ASR fusion.

The proposed system was trained using SpecAugment for data augmentation, 80-dimensional Mel-filterbanks, and CTC/attention hybrid loss for optimization. Pre-training with the AISHELL-2 dataset further improved performance and convergence speed.

2.10.2 Proposed System Model

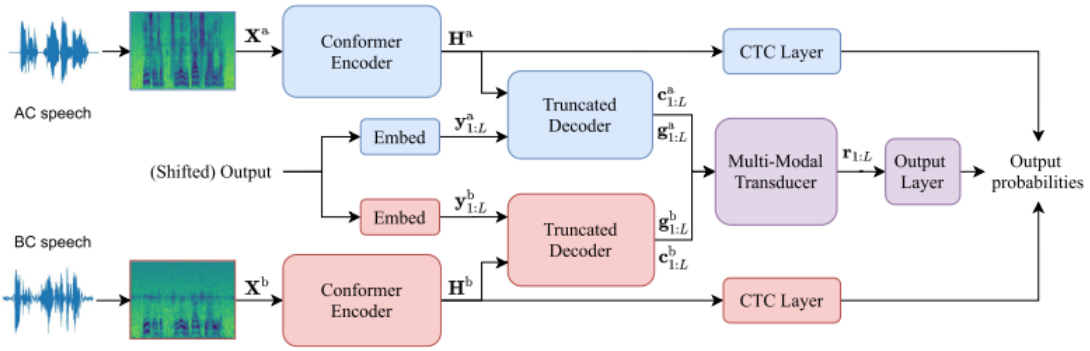

Figure 2.13: Architecture of the proposed end-to-end multi-modal ASR system.

2.10.3 Advantages

1. **Enhanced Noise Robustness:** Integrating BC speech allows the ASR to maintain high accuracy under low SNR conditions by utilizing noise-resistant features from bone conduction.
2. **Dynamic Fusion:** The Multi-Modal Transducer (MMT) adaptively adjusts the weighting between AC and BC modalities, optimizing performance according to real-time acoustic conditions.
3. **Large-Scale Dataset Contribution:** The ABCS corpus provides a benchmark resource for future research in bone-conducted and multi-modal speech processing.
4. **Improved Performance:** Experimental results show substantial reductions in Character Error Rate (CER) compared to both single-modal and simple feature-concatenation baselines.

2.10.4 Disadvantages

1. **Hardware Dependency:** The system requires specialized dual-microphone headsets for synchronized AC and BC data collection, which may limit scalability in general environments.
2. **High Computational Complexity:** The conformer and transformer-based architecture increases model size and training cost, potentially limiting real-time deployment.
3. **Limited High-Frequency Clarity:** Despite its noise resistance, BC speech loses important high-frequency information, affecting intelligibility in some phonemes.

Chapter 3

PROPOSED SYSTEM

The proposed system, **SIGN LINK: COMMUNICATION SERVICE FOR DEAF AND MUTE PEOPLE**, aims to transform the way hearing-impaired and non-impaired individuals communicate by integrating real-time sign language recognition, text, and speech translation into a single, accessible platform. The system provides a seamless communication bridge using intelligent video, audio, and natural language processing technologies to ensure inclusive, efficient, and meaningful interactions between users.

At its core, the application allows users to select their communication type-impaired or normal-during login, ensuring a personalized interaction experience. During a video call, the normal user can speak naturally while the impaired user receives live text captions or visual sign translations in real time. Conversely, when the hearing-impaired person communicates using sign language, the camera-based system detects and interprets their gestures, translating them into text and converting that text into speech for the other user. This two-way adaptive design ensures smooth and natural communication without the need for a human interpreter.

Additionally, SIGN LINK includes a real-time translation mode for face-to-face interactions. Users can simply point their camera toward someone using sign language and instantly receive the translated output in text or audio format. This functionality allows communication in dynamic, real-world situations-such as workplaces, public spaces, or social gatherings-promoting inclusivity and autonomy. The system leverages AI-based models for gesture recognition, speech understanding, and contextual translation, ensuring accurate, adaptive, and responsive communication support. Overall, SIGN LINK fosters inclusivity, accessibility, and independence within the hearing-impaired community while strengthening social interaction between all individuals.

3.1 Process Overview

1. **Project Planning**

   (a) **Define Project Scope and Objectives:** Clearly outline the overall purpose, goals, and intended outcomes of the SIGN LINK system, focusing on enabling real-time, bidirectional communication between impaired and non-impaired users.

   (b) **Identify Target Audience:** Understand the communication preferences and accessibility needs of both hearing-impaired and normal users to design a user-centric interface.

   (c) **Establish Project Timeline and Milestones:** Create a structured plan with milestones for data collection, design, model training, integration, testing, and deployment phases.

   (d) **Assemble Project Team:** Form a cross-functional team with expertise in artificial intelligence, machine learning, computer vision, mobile development, and accessibility design to ensure multidisciplinary input.

2. **Requirements Gathering and Analysis**

   (a) **Gather User Requirements:** Conduct interviews, surveys, and observation studies with hearing-impaired users to identify essential features and usability needs.

   (b) **Analyze User Feedback:** Evaluate collected feedback to refine key functionalities such as sign-to-text conversion accuracy, real-time responsiveness, and interface intuitiveness.

   (c) **Define Functional and Non-functional Requirements:** Specify system capabilities including gesture recognition, speech-to-text conversion, latency thresholds, data privacy, and reliability under varying network conditions.

3. **Design and Development**

   (a) **App Design:** Develop a visually accessible and intuitive UI/UX layout optimized for both normal and impaired users, with high-contrast visuals, large icons, and simple navigation.

   (b) **AI Model Integration:** Implement deep learning models (e.g., CNN and RNN-based architectures) for real-time sign recognition using live camera feeds.

   (c) **Speech and Text Conversion:** Integrate speech-to-text APIs and neural text-to-speech systems to enable clear and natural audio output.

   (d) **Software Development:** Build the mobile application using modern frameworks, ensuring smooth synchronization between video, audio, and translation modules.

(e) **Data Management:** Establish secure data pipelines for storing training data, managing user preferences, and protecting sensitive user interactions.

4. **Testing and Quality Assurance**

(a) **Unit Testing:** Test individual modules-such as video recognition, language conversion, and text rendering-for functionality and stability.

(b) **Integration Testing:** Validate the seamless interaction between different modules to ensure accurate, real-time processing without lag or frame loss.

(c) **User Acceptance Testing (UAT):** Involve real users, including deaf and mute individuals, to test usability, accessibility, and satisfaction with the communication experience.

(d) **Performance Testing:** Assess app responsiveness, processing speed, and accuracy under real-world network and hardware constraints.

(e) **Security and Compliance Testing:** Ensure that all data transmissions-especially video and audio-are encrypted and comply with privacy standards.

5. **Deployment and Maintenance**

(a) **App Store Deployment:** Launch the SIGN LINK application on Google Play and Apple App Store for broad accessibility and feedback collection.

(b) **Monitoring and Support:** Continuously track performance metrics, gather user reviews, and update AI models to improve accuracy and reliability.

(c) **Security Updates:** Regularly release patches to ensure data protection and address potential vulnerabilities.

(d) **Feature Enhancements:** Gradually expand capabilities with support for multiple sign languages, enhanced translation quality, emotion detection, and cloud-based synchronization for multi-device access.

3.1.1 Process Flow Diagram

Figure 3.1: Process Flow Diagram

3.1.2 Architecture Diagram

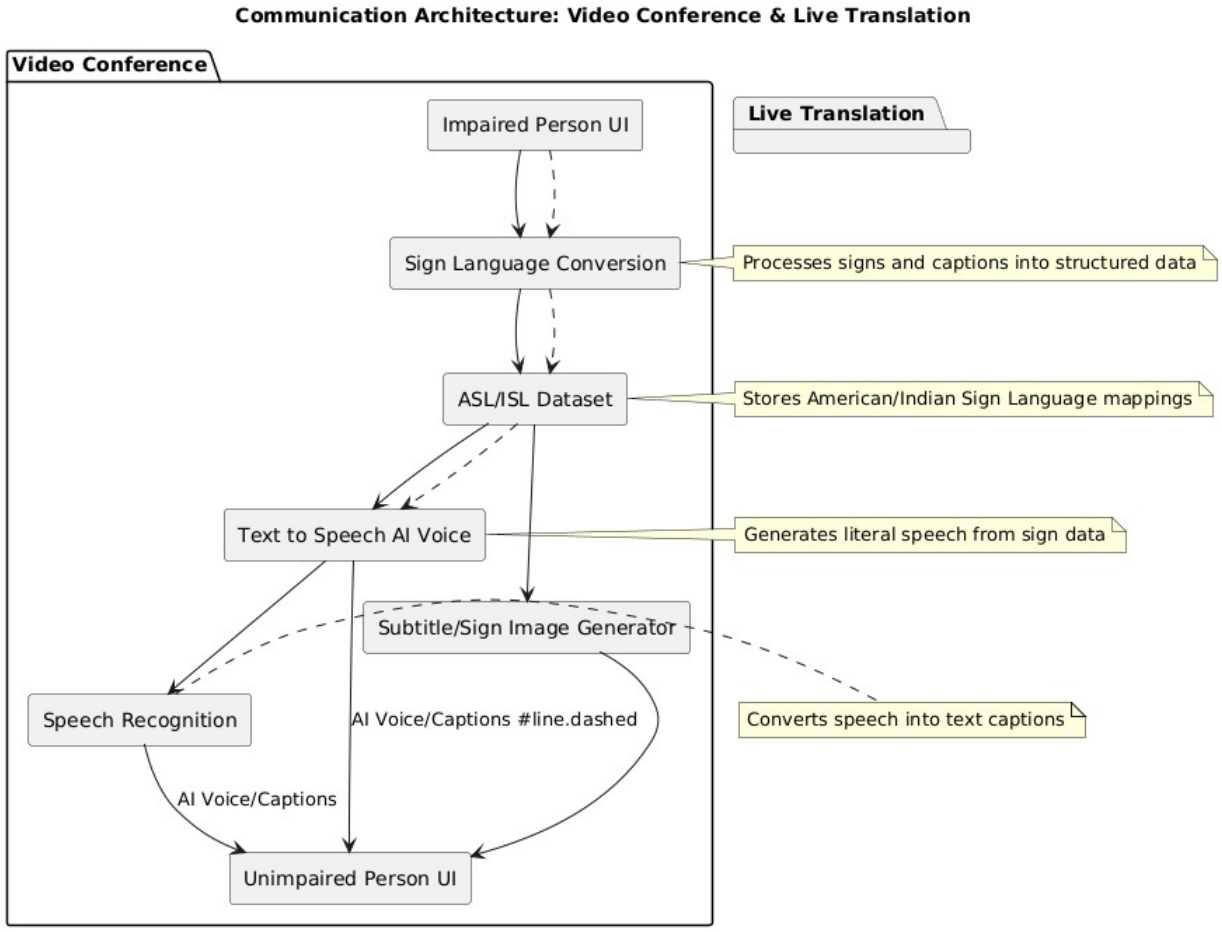

Figure 3.2: Architecture Diagram

3.1.3 Use case Diagram

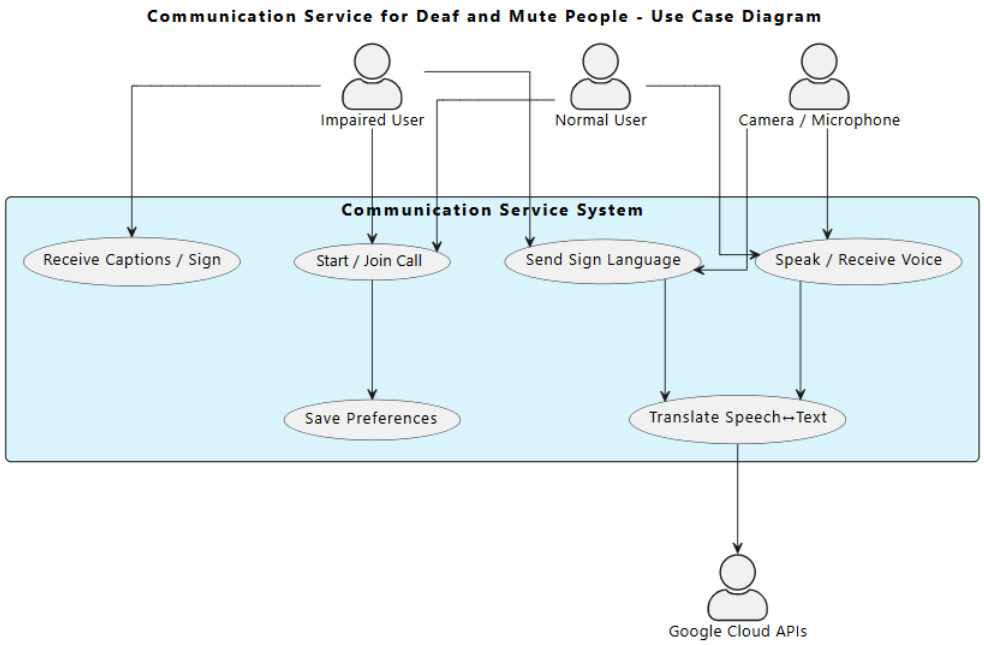

Figure 3.3: Use case diagram

3.1.4 Data Flow Diagram - Video Call Interaction

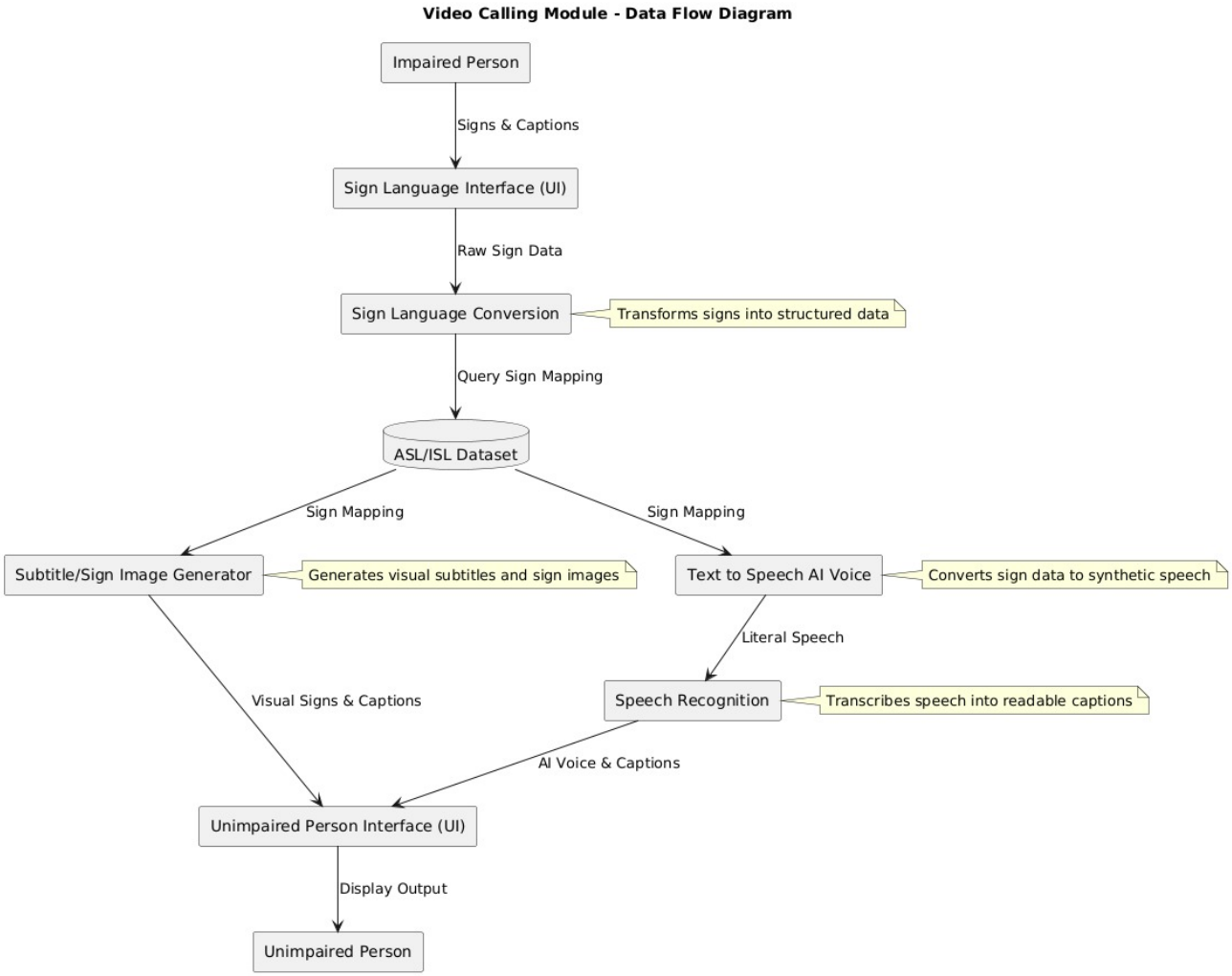

Figure 3.4: Data Flow Diagram - Video Call Interaction

3.1.5 Data Flow Diagram - Live Translation

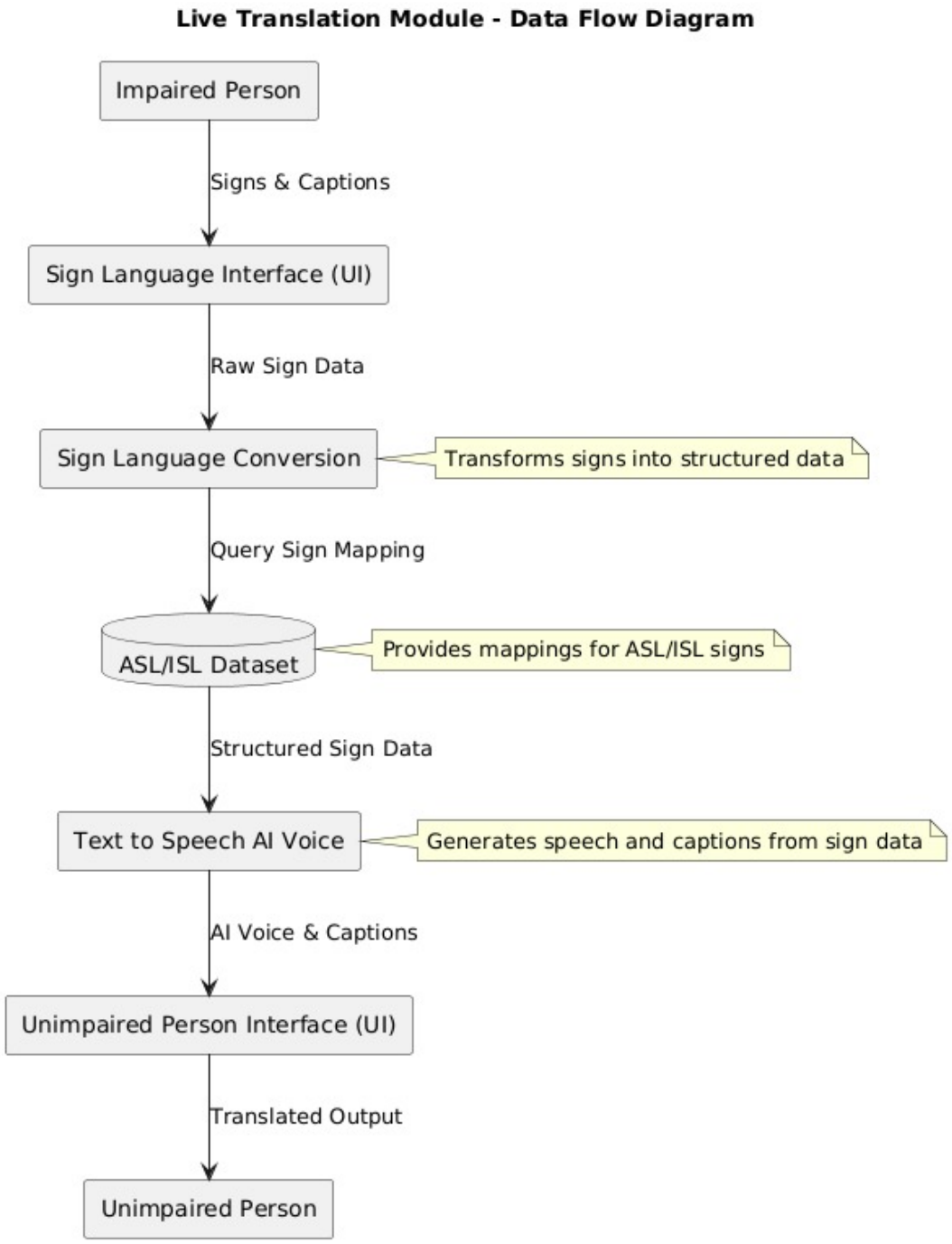

Figure 3.5: Data Flow Diagram - Live Translation

3.1.6 Class Diagram

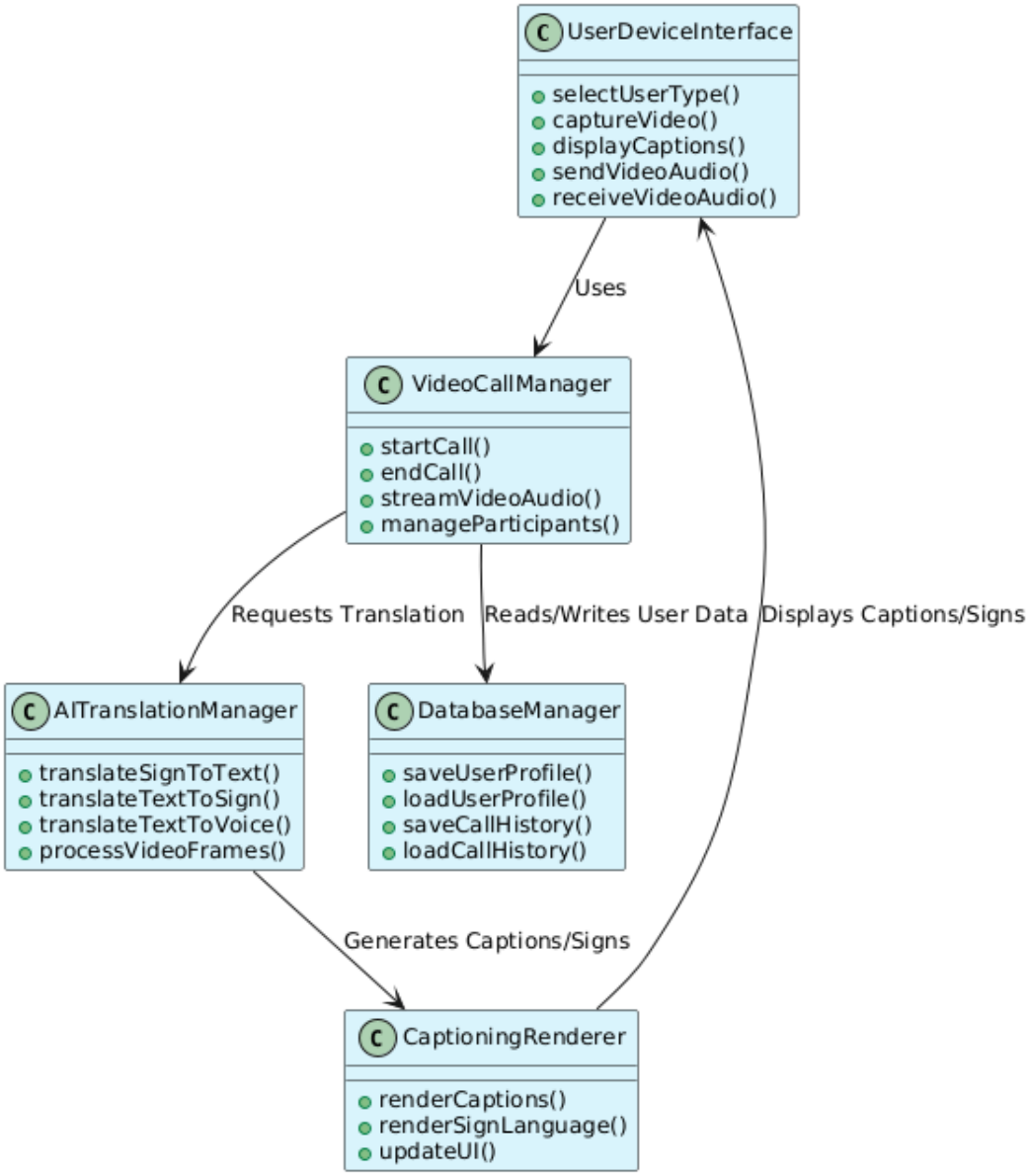

Figure 3.6: Class diagram

3.2 System Overview

The system architecture of “SignLink” is designed to provide real-time video communication enhanced with sign language recognition and translation features. The application enables deaf and hard-of-hearing users to communicate effectively through video calls integrated with Indian Sign Language (ISL) recognition, with optional support for American Sign Language (ASL).

The system is built using a web-based architecture where Node.js and Express act as the back-end server, while WebRTC enables peer-to-peer video communication. MediaPipe Hands is used for real-time hand landmark detection, and TensorFlow.js performs machine learning inference directly in the browser. Firebase Firestore is optionally used for storing gesture datasets and labels,

while trained models are stored locally in the browser for fast execution.

The overall architecture ensures low latency, real-time prediction, and seamless communication between users without requiring heavy server-side computation.

3.2.1 WebRTC Integration

SignLink integrates WebRTC (Web Real-Time Communication) to enable peer-to-peer audio and video communication between users. Socket.io is used as a signaling mechanism to exchange session descriptions (offer/answer) and ICE candidates required to establish a secure connection.

The system enforces HTTPS to ensure secure access to camera and microphone devices. Once connected, media streams are directly transmitted between peers, reducing server load and ensuring minimal latency. Additional real-time events such as chat messages, sign predictions, speech transcripts, and emoji reactions are also transmitted using Socket.io channels.

3.2.2 Application Main Interface

The landing page of SignLink provides users with multiple operational modes, including Video Call Mode, Live Translation Mode, and Training Studio.

In Video Call Mode, users can create or join rooms using unique room IDs. The interface includes video panels, chat sections, toggle buttons for microphone and camera, and controls for enabling speech-to-text (STT) or text-to-speech (TTS).

High-level UI components are built using HTML, CSS, and JavaScript, enhanced with Material Icons and responsive design techniques to ensure usability across devices.

3.2.3 Graphics and Interface Design

The user interface is designed to be clean, accessible, and responsive. The layout prioritizes clarity to ensure ease of use for individuals with hearing impairments. Visual indicators such as audio level meters, prediction labels, and animated emoji reactions enhance user interaction.

MediaPipe visualizations display real-time hand landmark points, helping users understand gesture detection accuracy. The design emphasizes simplicity, contrast, and readability to make communication intuitive and efficient.

3.2.4 System Modes and Functional Modules

Video Call Mode

In Video Call Mode, users connect through WebRTC and communicate via live video and audio. Simultaneously, MediaPipe detects hand landmarks from the camera feed and converts them into a 63-element feature vector.

This vector is passed to a trained TensorFlow.js model for prediction. Recognized signs are displayed as text and optionally broadcasted to the connected peer. Speech-to-text functionality converts spoken words into live captions, and text-to-speech converts recognized signs into audible output.

Live Translation Mode

The Live Translation Mode enables direct conversion between sign language and speech. In sign-to-text mode, the system detects gestures and displays translated text in real time. In speech-to-sign mode, spoken input is transcribed using the Web Speech API and displayed for interpretation.

Prediction smoothing techniques are applied to reduce noise and improve accuracy. The system supports switching between ISL and ASL models depending on user preference.

Training Studio

The Training Studio allows users to collect custom gesture datasets and train machine learning models directly in the browser. Users capture hand landmarks while labeling gestures. The collected dataset is stored temporarily and can be exported as JSON.

A dense neural network model is trained using TensorFlow.js and saved to local storage. The system also supports importing external datasets generated using Python-based preprocessing scripts. This feature enables extensibility and personalization of sign recognition models.

3.2.5 Machine Learning Pipeline

The machine learning pipeline begins with real-time hand detection using MediaPipe Hands, which extracts 21 hand landmarks, resulting in a 63-dimensional feature vector (x, y, z coordinates).

The data undergoes preprocessing to achieve translation and scale invariance. The processed vector is then passed to a dense neural network model trained either in Python using Keras or directly in the browser using TensorFlow.js.

The trained model is exported in TensorFlow.js format and stored locally for real-time inference. This ensures low-latency predictions without requiring server-side processing.

3.2.6 Data Persistence and Communication

Gesture labels and datasets can be stored in Firebase Firestore for cross-session persistence. Trained models and label mappings are stored in the browser's localStorage to allow offline usage.

Socket.io is used for transmitting prediction results, chat messages, audio levels, and custom

interaction events between peers. This ensures synchronized communication and enhanced collaborative experience.

3.3 System Requirements

3.3.1 Hardware Requirements

To ensure smooth operation of SignLink, the following hardware specifications are recommended:

Minimum RAM: 4 GB Processor: Dual-core 2.0 GHz or higher Camera: Functional webcam (720p recommended) Microphone: Integrated or external microphone Storage: At least 2 GB free space Internet: Stable broadband connection for real-time video communication

For machine learning training and dataset processing, a system with 8 GB RAM is recommended.

3.3.2 Software Requirements

SignLink runs in modern web browsers that support WebRTC and WebGL. Recommended browsers include Google Chrome and Microsoft Edge.

Backend Requirements: Node.js (v16 or higher) Express.js Socket.io

Frontend Requirements: MediaPipe Hands TensorFlow.js Firebase SDK (for Firestore)

For offline model training and dataset conversion: Python 3.8+ TensorFlow / Keras OpenCV MediaPipe Python Library

3.3.3 Node.js

Node.js is a JavaScript runtime environment used to build the backend server of SignLink. It handles static file serving, HTTPS redirection, and WebRTC signaling through Socket.io. Its event-driven architecture ensures efficient handling of multiple real-time connections.

3.3.4 TensorFlow.js

TensorFlow.js is a JavaScript library used for building and deploying machine learning models directly in the browser. It enables real-time inference for sign language recognition without requiring server-side computation. This improves privacy and reduces latency.

3.3.5 MediaPipe Hands

MediaPipe Hands is a real-time hand tracking solution developed by Google. It detects 21 hand landmarks per frame and provides precise 3D coordinates. These landmarks serve as input features

for the machine learning model used in gesture recognition.

3.3.6 Firebase Firestore

Firebase Firestore is a cloud-based NoSQL database used to store gesture labels and datasets. It allows persistent storage across sessions and supports synchronization between users when required.

3.3.7 Visual Studio Code

Visual Studio Code is used as the primary development environment for building the SignLink application. It provides syntax highlighting, debugging support, Git integration, and extension support for JavaScript and Python development.

3.4 Methodology

Upon launching the application in a web browser, the user is presented with a landing page offering different operational modes. If the user selects Video Call Mode, they either create or join a room using a unique room ID.

Once connected, WebRTC establishes a peer-to-peer video stream between participants. Simultaneously, the camera feed is processed by MediaPipe Hands to extract landmark coordinates. These coordinates are preprocessed and fed into a trained TensorFlow.js model for gesture prediction.

Recognized signs are displayed as text and optionally converted into speech using the Web Speech API. Speech input from users can also be converted into text captions.

If the user selects Training Studio, they can capture gesture samples, label them, train a neural network model in-browser, and save the model locally. The trained model can then be used in Video Call or Translation Mode.

At any point, users can switch modes, retrain models, or load saved models, ensuring flexibility and extensibility of the system.

Chapter 4

RESULTS

4.1 Case Study: Enhancing Communication Accessibility Using Real-Time Sign Language Recognition

Traditional communication platforms often fail to fully support deaf and hard-of-hearing individuals due to their heavy reliance on spoken language. Although video calling allows visual interaction, it does not inherently provide real-time sign language translation or accessible speech transcription. SignLink was developed to address these challenges by integrating real-time sign language recognition, speech-to-text, and text-to-speech capabilities into a unified video communication platform.

One of the major barriers in conventional communication systems is the absence of automatic sign interpretation. Deaf users must rely entirely on the other participant's knowledge of sign language. SignLink enhances accessibility by incorporating MediaPipe-based hand tracking and TensorFlow.js-powered gesture recognition directly within the browser. This allows users to perform Indian Sign Language (ISL) gestures that are instantly translated into readable text. Optional American Sign Language (ASL) support further increases the system's flexibility.

In Video Call Mode, two users can connect through WebRTC-based peer-to-peer communication. During the session, the system continuously captures hand landmark coordinates from the webcam feed. These landmarks are processed into a 63-dimensional feature vector and passed to a trained neural network model. The predicted gesture is displayed as text and can optionally be converted into speech using the Web Speech API. This feature enables seamless interaction between a sign-language user and a non-sign-language user.

The Live Translation Mode demonstrates the system's standalone translation capabilities. In sign-to-text mode, gestures are recognized and displayed in real time. In speech-to-sign mode, spoken words are converted into text captions, allowing deaf users to understand verbal communication

without relying on external interpreters. Prediction smoothing mechanisms were implemented to reduce noise and improve classification stability during continuous hand movements.

The Training Studio module enables customization and scalability of the system. Users can capture gesture samples, label them, and train a neural network directly within the browser using TensorFlow.js. The trained model is stored locally and can be immediately deployed for real-time inference. This significantly reduces dependency on pre-trained static datasets and allows adaptation to different sign variations or regional dialects.

Performance evaluation indicates that browser-based inference provides low-latency predictions suitable for real-time communication. Since the machine learning model runs on the client side, server load is minimized and user privacy is enhanced. Additionally, optional Firebase Firestore integration allows persistent storage of gesture labels and datasets across sessions.

Unlike traditional video conferencing platforms that only support audio and video streams, SignLink integrates machine learning, computer vision, and real-time communication technologies into a single accessible system. The combination of WebRTC, MediaPipe, TensorFlow.js, and Web Speech APIs creates an inclusive communication environment that improves interaction quality, reduces communication barriers, and promotes digital accessibility.

Overall, the results demonstrate that SignLink successfully enhances communication experiences by providing real-time gesture recognition, multimodal translation, and adaptive training capabilities. The system proves that accessible communication tools can be efficiently implemented using modern web technologies without requiring specialized hardware or external interpretation services.

Chapter 5

CONCLUSION

In conclusion, the proposed **SIGN LINK: Communication Service for Deaf and Mute People** represents a significant step forward in creating an inclusive communication ecosystem that bridges the gap between hearing-impaired and normal users. By integrating advanced technologies such as sign language recognition, speech-to-text, text-to-speech, and live translation services, the system facilitates seamless, real-time, and natural interaction among users of all abilities.

The platform's real-time features - including live captions, sign-to-text conversion, text-to-voice output, and camera-based in-person translation - empower users with efficient and accessible tools for daily communication. These functionalities not only enhance user experience but also foster independence and social inclusion for individuals with hearing or speech impairments.

Moreover, the application incorporates a user-friendly interface and intuitive video calling environment, ensuring simplicity and ease of use. Its AI-driven translation modules continuously learn and adapt, providing improved accuracy and fluency over time. Through these capabilities, SIGN LINK enables effortless collaboration between impaired and non-impaired users, minimizing misunderstandings and promoting meaningful engagement.

Beyond immediate communication benefits, the system demonstrates the potential of assistive technology to transform accessibility on a broader scale. It lays the groundwork for future enhancements such as multi-language sign support, emotion recognition, and integration with wearable devices for continuous, context-aware interaction.

Overall, SIGN LINK offers a practical, scalable, and human-centered solution to one of the most persistent barriers in modern communication - ensuring that no individual is excluded from expressing themselves or being understood. This work underscores the vital role of technology in fostering inclusivity, accessibility, and equal participation in everyday life.

References

[1] H. -N. Vu, T. Hoang, C. Tran and C. Pham, ”Sign Language Recognition With Self-Learning Fusion Model,” in IEEE Sensors Journal, vol. 23, no. 22, pp. 27828-27840, 15 Nov.15, 2023.

[2] Y. Wen, Z. Zhang, J. Sun, J. Li, C. S. Chen and G. Niu, ”SAW: Semantic-Aware WebRTC Transmission Using Diffusion-Based Scalable Video Coding,” in IEEE Internet of Things Journal, vol. 12, no. 5, pp. 5346-5359, 1 March1, 2025.

[3] Yuankang Zhao, Qinghua Wu, Gerui Lv, Furong Yang, Jiuhai Zhang, Feng Peng, Yanmei Liu, Zhenyu Li, Hongyu Guo, Ying Chen, and Gaogang Xie. 2025. Understanding and Taming the Inflated Latency in Mobile Cloud Rendering. ACM Trans. Multimedia Comput. Commun. Appl. Just Accepted (July 2025).

[4] H. Hu, J. Pu, W. Zhou and H. Li, ”Collaborative Multilingual Continuous Sign Language Recognition: A Unified Framework,” in IEEE Transactions on Multimedia, vol. 25, pp. 7559-7570, 2023.

[5] W. Zhao, H. Hu, W. Zhou, Y. Mao, M. Wang and H. Li, ”MASA: Motion-Aware Masked Autoencoder With Semantic Alignment for Sign Language Recognition,” in IEEE Transactions on Circuits and Systems for Video Technology, vol. 34, no. 11, pp. 10793-10804, Nov. 2024.

[6] D. Sánchez Ruiz, J. A. Olvera-López and I. Olmos-Pineda, ”Word Level Sign Language Recognition via Handcrafted Features,” in IEEE Latin America Transactions, vol. 21, no. 7, pp. 839-848, July 2023.

[7] H. Hu, J. Pu, W. Zhou, H. Fang and H. Li, ”Prior-Aware Cross Modality Augmentation Learning for Continuous Sign Language Recognition,” in IEEE Transactions on Multimedia, vol. 26, pp. 593-606, 2024.

[8] T. Han, Q. Yang, Z. Shi, S. He and Z. Zhang, ”Semantic-Preserved Communication System for Highly Efficient Speech Transmission,” in IEEE Journal on Selected Areas in Communications, vol. 41, no. 1, pp. 245-259, Jan. 2023.

[9] W. Zhou, W. Zhao, H. Hu, Z. Li and H. Li, ”Scaling up Multimodal Pre-Training for Sign Language Understanding,” in IEEE Transactions on Pattern Analysis and Machine Intelligence.

[10] M. Wang, J. Chen, X. -L. Zhang and S. Rahardja, ”End-to-End Multi-Modal Speech Recognition on an Air and Bone Conducted Speech Corpus,” in IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 31, pp. 513-524, 2023.

Appendix A

Code

A.1 Translation.js

```
const videoElement = document.getElementById('input-video');
const canvasElement = document.getElementById('output-canvas');
const canvasCtx = canvasElement.getContext('2d');
const signView = document.getElementById('sign-view');
const speechView = document.getElementById('speech-view');
const speechPanel = document.getElementById('speech-panel');
const speechCaptionLog = document.getElementById('speech-caption-log');
const camBtn = document.getElementById('cam-btn');
const ttsBtn = document.getElementById('tts-btn');
const sttResult = document.getElementById('stt-result');
const listeningText = document.getElementById('listening-text');
let signVoiceToggle = document.getElementById('sign-voice-toggle');
const voiceSubtitles = document.getElementById('voice-subtitles');

let isSignMode = true; // true = sign detection, false = voice recognition
let isCamOn = true;
let isTTSOn = true;
let lastSpokenLabel = "";
let lastSpokenTime = 0;
let localStream = null;
let camera = null;
let cameraLoopId = null;
let recognition = null; // For Speech to Text
```

```
// --- Spelling Mode State ---
let accumulatedWord = "";
let lastLetterTime = 0;
let lastAddedLetter = null;
let spellingInterval = null;
const SPELLING_IDLE_TIMEOUT_MS = 5000;

// --- Model & State ---
// Hybrid Model Approaches:
// 1. Server Model (Pre-trained ISL Dataset)
// 2. Local Model (User Generated via AI Training)
let serverModel = null;
let serverLabels = [];
let localModel = null;
let localLabels = [];

// Dynamic sign support
let localModelDynamic = null;
let localLabelsDynamic = [];
let dynamicLabelHandRequirements = {};
let dynamicFrameBuffer = [];
const MAX_DYNAMIC_FRAMES = 30;
const DYNAMIC_ANALYZE_MS = 1500;
let dynamicBufferStartTime = 0;
const BIG_MOTION_CHANGE_THRESHOLD = 0.06;
const ASL_Z_MIN_CONFIDENCE = 0.82;
const ASL_Z_MIN_FRAMES = 6;
const ASL_Z_MIN_X_RANGE = 0.06;
const ASL_Z_MIN_Y_RANGE = 0.015;
const ASL_Z_MIN_PATH_DISTANCE = 0.12;
const ASL_Z_MIN_HORIZONTAL_TRAVEL = 0.09;
const ASL_Z_MIN_VERTICAL_TRAVEL = 0.02;
const ASL_Z_MIN_DIRECTION_CHANGES = 1;
const ASL_Z_MIN_CURVATURE_RATIO = 1.03;
const ASL_Z_MIN_WHOLE_HAND_PATH = 0.11;
```

```
const ASL_Z_MIN_ACTIVE_LANDMARK_RATIO = 0.28;
let lastDisplayedPrediction = null;
let lastDisplayedFrame = null;
const STATIC_STILL_DURATION_MS = 1000;
const MOTION_THRESHOLD = 0.02;
let previousMotionFrame = null;
let staticStillStartTime = 0;
const NO_HANDS_TIMEOUT_MS = 2000;
let lastHandDetectedTime = Date.now();
let noHandsTimeoutId = null;

const predictionBuffer = [];
let localStorageModelKey = 'my-isl-model'; // Default
let localStorageLabelKey = 'isl_labels';

function normalizeAlphabetLabel(label) {
    if (typeof label !== 'string') return label;
    return /^[a-zA-Z]$/.test(label) ? label.toUpperCase() : label;
}

function normalizeLabelList(labels) {
    let changed = false;
    const normalized = (labels || []).map((label) => {
        const nextLabel = normalizeAlphabetLabel(label);
        if (nextLabel !== label) changed = true;
        return nextLabel;
    });
    return { labels: normalized, changed };
}

function normalizeHandRequirementMap(map) {
    let changed = false;
    const normalized = {};

    Object.entries(map || {}).forEach(([label, requirement]) => {
        const nextLabel = normalizeAlphabetLabel(label);
```

```
        if (nextLabel !== label) changed = true;
        normalized[nextLabel] = requirement;
    });

    return { map: normalized, changed };
}

// Language Selector Logic
const langSelect = document.getElementById('lang-select');
if (langSelect) {
    // Sync state on page load in case browser restores Dropdown state
    if (langSelect.value === 'ASL') {
        localStorageModelKey = 'my-asl-model';
        localStorageLabelKey = 'asl_labels';
    } else {
        localStorageModelKey = 'my-isl-model';
        localStorageLabelKey = 'isl_labels';
    }

    langSelect.addEventListener('change', (e) => {
        const lang = e.target.value;
        if (lang === 'ISL') {
            localStorageModelKey = 'my-isl-model';
            localStorageLabelKey = 'isl_labels';
        } else {
            // For ASL or others, we might only have local models for now
            // or different server models. For now, assume mainly local for ASL.
            localStorageModelKey = 'my-asl-model';
            localStorageLabelKey = 'asl_labels';
        }
        sttResult.innerText = `Switched to ${lang}. Loading models...`;
        loadSavedModelAndLabels();
    });
}

// TTS starts enabled by default for live translation
```

```
if (ttsBtn) {
    ttsBtn.innerHTML = '<span class="material-icons">volume_up</span>';
    ttsBtn.classList.remove('red-btn');
}

// Load Models and Labels (Hybrid)
async function loadSavedModelAndLabels() {
    try {
        // Reset State
        serverModel = null;
        serverLabels = [];
        localModel = null;
        localLabels = [];
        localModelDynamic = null;
        localLabelsDynamic = [];
        dynamicLabelHandRequirements = {};
        predictionBuffer.length = 0;
        dynamicFrameBuffer = [];
        dynamicBufferStartTime = 0;
        lastDisplayedPrediction = null;
        lastDisplayedFrame = null;

        const promises = [];

        // 1. Load Server Model
        const serverLoad = async () => {
            console.log("Attempting to load Server Model...");
            try {
                const isASL = localStorageModelKey === 'my-asl-model';
                const modelPath = isASL ? 'model/asl/model.json' :
                'model/model.json';
                const labelsPath = isASL ? 'model/asl/labels.json' :
                'labels.json';

                const response = await fetch(labelsPath);
                if (response.ok) {
```

```
            serverLabels = normalizeLabelList(await
            response.json()).labels;
            try {
                serverModel = await tf.loadLayersModel(modelPath);
                console.log(`Server Model loaded
                (${serverLabels.length} labels from
                ${labelsPath})`);
            } catch (tfErr) {
                console.error("TFJS Server Model Load
                Error:", tfErr);
                serverModel = null;
            }
        } else {
            console.warn(`${labelsPath} not found.`);
        }
    } catch (e) {
        console.error("Server model load failed fatally:", e);
        serverModel = null;
    }
    return Promise.resolve();
};
promises.push(serverLoad());

// 2. Load Local Static Model
const localLoad = async () => {
    console.log("Attempting to load Local Static Model...");
    try {
        const localLabelData =
        localStorage.getItem(`${localStorageLabelKey}-static`);
        if (localLabelData) {
            const normalizedLocalLabels =
            normalizeLabelList(JSON.parse(localLabelData));
            localLabels = normalizedLocalLabels.labels;
            if (normalizedLocalLabels.changed) {
                localStorage.setItem(`${localStorageLabelKey}-
                static`, JSON.stringify(localLabels));
```

```
            }
            console.log(‘Diagnostic -> Loaded Local Static Labels
            for ${localStorageLabelKey}:‘, localLabels);
            try {
                localModel = await
                tf.loadLayersModel(‘localstorage://${
                localStorageModelKey}-static‘);
                console.log(‘Local Static Model loaded
                (${localLabels.length} labels)‘);
            } catch (e) {
                console.warn("Local static model weights not found
                in localStorage.");
                localModel = null;
            }
        }
    } catch (e) {
        console.warn("Local static model load failed:", e);
        localModel = null;
    }
    return Promise.resolve(); // NEVER reject because we want
    server model to survive
};
promises.push(localLoad());

// 3. Load Local Dynamic Model
const dynamicLoad = async () => {
    console.log("Attempting to load Local Dynamic Model...");
    try {
        const dynamicLabelData =
        localStorage.getItem(‘${localStorageLabelKey}-dynamic‘);
        if (dynamicLabelData) {
            const normalizedDynamicLabels =
            normalizeLabelList(JSON.parse(dynamicLabelData));
            localLabelsDynamic = normalizedDynamicLabels.labels;
            if (normalizedDynamicLabels.changed) {
                localStorage.setItem(‘${localStorageLabelKey}-
```

```
                dynamic', JSON.stringify(localLabelsDynamic));
            }
            const dynamicReqData =
            localStorage.getItem('${localStorageLabelKey}-dynamic-
            hand-req');
            const normalizedHandReqs =
            normalizeHandRequirementMap(dynamicReqData ?
            JSON.parse(dynamicReqData) : {});
            dynamicLabelHandRequirements = normalizedHandReqs.map;
            if (normalizedHandReqs.changed) {
                localStorage.setItem('${localStorageLabelKey}-
                dynamic-hand-req',
                JSON.stringify(dynamicLabelHandRequirements));
            }
            try {
                localModelDynamic = await
                tf.loadLayersModel('localstorage://${
                localStorageModelKey}-dynamic');
                console.log('Local Dynamic Model loaded
                (${localLabelsDynamic.length} labels)');
            } catch (e) {
                console.warn("Local dynamic model weights not found
                in localStorage.");
                localModelDynamic = null;
            }
        }
    } catch (e) {
        console.warn("Local dynamic model load failed:", e);
    }
};
promises.push(dynamicLoad());

// Wait for all promises (use allSettled so one failure doesn't
kill others)
await Promise.allSettled(promises);
```

```
        // 4. UI Feedback - only show error if no models found
        const loadedModels = [];
        if (serverModel) loadedModels.push("Server");
        if (localModel) loadedModels.push("Local Static");
        if (localModelDynamic) loadedModels.push("Local Dynamic");

        // Don't show models loaded message - keep display clear
        if (loadedModels.length === 0) {
            sttResult.innerText = "No models found. Please train in AI
            Training mode.";
        }

        // "Go to Training" button if absolutely nothing
        if (!serverModel && !localModel && !localModelDynamic) {
            if (!document.getElementById('goto-training-btn')) {
                const btn = document.createElement('button');
                btn.id = 'goto-training-btn';
                btn.innerText = "Go to AI Training";
                btn.className = "control-btn";
                btn.style.marginTop = "10px";
                btn.style.background = "#3b82f6";
                btn.onclick = () => window.location.href = 'training.html';
                sttResult.parentElement.appendChild(btn);
            }
        } else {
            const btn = document.getElementById('goto-training-btn');
            if (btn) btn.remove();
        }

    } catch (e) {
        console.error("Error in hybrid load:", e);
        sttResult.innerText = "Error loading systems.";
    }
}
loadSavedModelAndLabels();
```

```
// --- MediaPipe Setup ---
const hands = new Hands({
    locateFile: (file) => `https://cdn.jsdelivr.net/npm/@mediapipe/hands/${file}`
});

hands.setOptions({
    maxNumHands: 2,
    modelComplexity: 1,
    minDetectionConfidence: 0.5,
    minTrackingConfidence: 0.5
});

hands.onResults(onResults);

function preprocessLandmarks(landmarks) {
    const wrist = landmarks[0];

    // 1. Translation Invariance
    let shifted = landmarks.map(p => ({
        x: p.x - wrist.x,
        y: p.y - wrist.y,
        z: p.z - wrist.z
    }));

    // 2. Scale Invariance
    const indexMCP = shifted[5];
    const distance = Math.sqrt(
        Math.pow(indexMCP.x, 2) +
        Math.pow(indexMCP.y, 2) +
        Math.pow(indexMCP.z, 2)
    ) || 1e-6;

    // 3. Normalize
    return shifted.flatMap(p => [
        p.x / distance,
        p.y / distance,
```

```
        p.z / distance
    ]);
}

function getSmoothedPrediction(predLabel) {
    predictionBuffer.push(predLabel);
    if (predictionBuffer.length > 10) predictionBuffer.shift(); // Slightly
    faster response
    const counts = {};
    predictionBuffer.forEach(l => counts[l] = (counts[l] || 0) + 1);
    return Object.entries(counts).sort((a, b) => b[1] - a[1])[0][0];
}

function updateMotionState(currentFrame) {
    const now = Date.now();

    if (!previousMotionFrame) {
        previousMotionFrame = [...currentFrame];
        staticStillStartTime = now;
        return { isStillFrame: false, stillForMs: 0 };
    }

    let totalDelta = 0;
    for (let i = 0; i < currentFrame.length; i++) {
        totalDelta += Math.abs(currentFrame[i] - previousMotionFrame[i]);
    }

    const motionScore = totalDelta / currentFrame.length;
    const isStillFrame = motionScore < MOTION_THRESHOLD;

    if (isStillFrame) {
        if (staticStillStartTime === 0) staticStillStartTime = now;
    } else {
        staticStillStartTime = 0;
    }
```

```
    previousMotionFrame = [...currentFrame];
    const stillForMs = staticStillStartTime ? (now - staticStillStartTime)
    : 0;
    return { isStillFrame, stillForMs };
}

function resetMotionState() {
    previousMotionFrame = null;
    staticStillStartTime = 0;
}

function getFrameDifference(frameA, frameB) {
    if (!frameA || !frameB || frameA.length !== frameB.length) return
    Infinity;

    let totalDelta = 0;
    for (let i = 0; i < frameA.length; i++) {
        totalDelta += Math.abs(frameA[i] - frameB[i]);
    }
    return totalDelta / frameA.length;
}

function updateDisplayedPrediction(label, conf, isDynamic, currentFrame) {
    lastDisplayedPrediction = { label, conf, isDynamic };
    lastDisplayedFrame = [...currentFrame];
}

function shouldKeepLastPrediction(currentFrame) {
    if (!lastDisplayedPrediction || !lastDisplayedFrame) return false;
    const diff = getFrameDifference(currentFrame, lastDisplayedFrame);
    return diff < BIG_MOTION_CHANGE_THRESHOLD;
}



function flipLandmarks(landmarks) {
    return landmarks.map(p => ({
```

```
        x: 1 - p.x,
        y: p.y,
        z: p.z
    }));
}

// Helper to run a single model prediction
function predictSingleModel(modelInstance, labels, tensor) {
    if (!modelInstance || !labels.length) return { label: null, conf: 0 };

    const pred = modelInstance.predict(tensor);
    const conf = pred.max().dataSync()[0];
    const idx = pred.argMax(-1).dataSync()[0];

    // Cleanup happens in tf.tidy in caller
    return { label: normalizeAlphabetLabel(labels[idx]), conf: conf };
}

function normalizeHandRequirement(rawValue) {
    if (rawValue === 1 || rawValue === '1') return 1;
    if (rawValue === 2 || rawValue === '2') return 2;
    return 'any';
}

function isASLDynamicSpellingLetter(label) {
    if (localStorageModelKey !== 'my-asl-model') return false;
    if (typeof label !== 'string') return false;
    return label.toUpperCase() === 'Z';
}

function getASLZMotionMetrics(frameBuffer) {
    const tipPoints = (frameBuffer || [])
        .filter(frame => Array.isArray(frame) && frame.length >= 26)
        .map(frame => ({ x: frame[24], y: frame[25] }));

    if (tipPoints.length < 2) {
```

```
        return {
            frameCount: tipPoints.length,
            xRange: 0,
            yRange: 0,
            pathDistance: 0,
            horizontalTravel: 0,
            verticalTravel: 0,
            directionChanges: 0,
            endToEndDistance: 0,
            curvatureRatio: 0
        };
    }
```

A.2 Training.js

```
// --- DOM Elements ---
const videoElement = document.getElementById('inputVideo');
const canvasElement = document.getElementById('outputCanvas');
const canvasCtx = canvasElement.getContext('2d');
const langSelect = document.getElementById('langSelect');
const labelInput = document.getElementById('labelInput');
const captureBtn = document.getElementById('captureBtn');
const trainBtn = document.getElementById('trainBtn');
const saveBtn = document.getElementById('saveBtn');
const statusMsg = document.getElementById('statusMsg');
const dataList = document.getElementById('dataList');
const totalSamplesBadge = document.getElementById('totalSamples');
const recIndicator = document.getElementById('recIndicator');
const uploadBtn = document.getElementById('uploadBtn');
const uploadInput = document.getElementById('uploadInput');
const revertBtn = document.getElementById('revertBtn');
const clearAllBtn = document.getElementById('clearAllBtn');
const testBtn = document.getElementById('testBtn');
const testResult = document.getElementById('testResult');
```

```
const dataPanel = document.querySelector('.data-panel');
const openDataPanelBtn = document.getElementById('openDataPanelBtn');
const closeDataPanelBtn = document.getElementById('closeDataPanelBtn');
const drawerBackdrop = document.getElementById('drawerBackdrop');

// Sign Card Elements
const signCardBtn = document.getElementById('signCardBtn');
const signCardInput = document.getElementById('signCardInput');
const signCardStatus = document.getElementById('signCardStatus');
const clearSignDetailsBtn = document.getElementById('clearSignDetailsBtn');
const signCardFileName = document.getElementById('signCardFileName');

// Dynamic mode elements
const staticModeBtn = document.getElementById('staticModeBtn');
const dynamicModeBtn = document.getElementById('dynamicModeBtn');
const modeDescription = document.getElementById('modeDescription');
const captureHint = document.getElementById('captureHint');
const dynamicControls = document.getElementById('dynamicControls');
const startRecordBtn = document.getElementById('startRecordBtn');
const stopRecordBtn = document.getElementById('stopRecordBtn');
const frameCounter = document.getElementById('frameCounter');
const frameCount = document.getElementById('frameCount');
const recordingProgress = document.getElementById('recordingProgress');
const progressBar = document.getElementById('progressBar');

// --- State ---
let isCollecting = false;
let collectedData = [];
let currentLang = 'ISL';
let model = null;
let recordingMode = 'static'; // 'static' or 'dynamic'
const MAX_STATIC_SAMPLES_PER_SESSION = 100;
let staticSessionSampleCount = 0;
let isStaticPausedNoHands = false;

// Dynamic recording state
```

```
let isDynamicRecording = false;
let dynamicFrameBuffer = [];
const MAX_DYNAMIC_FRAMES = 30;
const TARGET_FPS = 10; // Capture ~10 frames per second
let lastFrameCaptureTime = 0;
let dynamicRecordingMaxHands = 1;

// Test mode state
let isTestMode = false;
let testStaticModel = null;
let testStaticLabels = [];
let testDynamicModel = null;
let testDynamicLabels = [];
let testDynamicFrameBuffer = [];
let testDynamicBufferStartTime = 0;
const TEST_DYNAMIC_ANALYZE_MS = 1200;

function normalizeLabel(label) {
    const trimmed = (label || '').trim();
    if (!trimmed) return '';
    if (/^[a-zA-Z]$/.test(trimmed)) return trimmed.toUpperCase();
    return trimmed;
}

function normalizeDatasetLabels(samples) {
    let changed = false;
    const normalized = samples.map((sample) => {
        const normalizedLabel = normalizeLabel(sample.label);
        if (normalizedLabel !== sample.label) {
            changed = true;
            return { ...sample, label: normalizedLabel };
        }
        return sample;
    });
    return { normalized, changed };
}
```

```
function getUntrainedSampleCount() {
    return collectedData.filter(sample => sample.isTrained ===
    false).length;
}

function updateRevertButtonState() {
    if (!revertBtn) return;

    const untrainedCount = getUntrainedSampleCount();
    revertBtn.disabled = untrainedCount === 0;
    revertBtn.innerHTML = `<span class="material-icons">undo</span>Revert
    New Data${untrainedCount ? ` (${untrainedCount})` : ''}`;
}

// Storage Keys
const STORAGE_KEYS = {
    'ISL': { model: 'my-isl-model', labels: 'isl_labels', data: 'isl_data'
    },
    'ASL': { model: 'my-asl-model', labels: 'asl_labels', data: 'asl_data' }
};

// --- Initialization ---
async function init() {
    startCamera();
    await loadDataFromServer();
    checkForSavedModels(); // Check if models already exist in localStorage
    renderDataList();
    updateRevertButtonState();
    setupModeToggle();
    setupMobileDataDrawer();
}

function setupMobileDataDrawer() {
    if (!dataPanel) return;
```

```
    const openDrawer = () => {
        dataPanel.classList.add('open');
        if (drawerBackdrop) drawerBackdrop.classList.add('active');
    };

    const closeDrawer = () => {
        dataPanel.classList.remove('open');
        if (drawerBackdrop) drawerBackdrop.classList.remove('active');
    };

    if (openDataPanelBtn) {
        openDataPanelBtn.addEventListener('click', openDrawer);
    }
    if (closeDataPanelBtn) {
        closeDataPanelBtn.addEventListener('click', closeDrawer);
    }
    if (drawerBackdrop) {
        drawerBackdrop.addEventListener('click', closeDrawer);
    }

    window.addEventListener('keydown', (event) => {
        if (event.key === 'Escape') closeDrawer();
    });

    window.addEventListener('resize', () => {
        if (window.innerWidth > 980) {
            closeDrawer();
        }
    });
}

// Check if models are already saved in localStorage
function checkForSavedModels() {
    const staticLabels =
    localStorage.getItem(`${STORAGE_KEYS[currentLang].labels}-static`);
    const dynamicLabels =
```

```
    localStorage.getItem(`${STORAGE_KEYS[currentLang].labels}-dynamic`);

    if (staticLabels || dynamicLabels) {
        let modelInfo = "Saved models found: ";
        if (staticLabels) modelInfo += "Static  ";
        if (dynamicLabels) modelInfo += "Dynamic ";
        statusMsg.innerText = ` ${modelInfo}. You can use these in Live
        Translation!`;
        saveBtn.disabled = true; // Models already saved
    }
}

// Mode toggle setup
function setupModeToggle() {
    staticModeBtn.addEventListener('click', () => switchMode('static'));
    dynamicModeBtn.addEventListener('click', () => switchMode('dynamic'));

    startRecordBtn.addEventListener('click', startDynamicRecording);
    stopRecordBtn.addEventListener('click', stopDynamicRecording);
}

function switchMode(mode) {
    recordingMode = mode;
    testDynamicFrameBuffer = [];
    testDynamicBufferStartTime = 0;

    // Update button states
    staticModeBtn.classList.toggle('active', mode === 'static');
    dynamicModeBtn.classList.toggle('active', mode === 'dynamic');

    // Update UI visibility
    if (mode === 'static') {
        captureBtn.style.display = 'flex';
        captureHint.style.display = 'block';
        dynamicControls.style.display = 'none';
        modeDescription.textContent = 'Static: Single pose signs (A, B,
```

```
        Hello, etc.)';
    } else {
        captureBtn.style.display = 'none';
        captureHint.style.display = 'none';
        dynamicControls.style.display = 'block';
        modeDescription.textContent = 'Dynamic: Movement signs (Thank You,
        Please, Sorry, etc.)';
    }
}

function setTestResult(text) {
    if (testResult) testResult.textContent = text;
}

async function loadTestModels() {
    testStaticModel = null;
    testStaticLabels = [];
    testDynamicModel = null;
    testDynamicLabels = [];

    if (model?.static && Array.isArray(model.staticLabels) &&
    model.staticLabels.length) {
        testStaticModel = model.static;
        testStaticLabels = model.staticLabels;
    }
    if (model?.dynamic && Array.isArray(model.dynamicLabels) &&
    model.dynamicLabels.length) {
        testDynamicModel = model.dynamic;
        testDynamicLabels = model.dynamicLabels;
    }

    if (!testStaticModel) {
        const savedStaticLabels =
        localStorage.getItem(`${STORAGE_KEYS[currentLang].labels}-static`);
        if (savedStaticLabels) {
            testStaticLabels = JSON.parse(savedStaticLabels);
```

```
            try {
                testStaticModel = await
                tf.loadLayersModel(`localstorage://${STORAGE_KEYS[currentLan
                g].model}-static`);
            } catch (e) {
                console.warn('Unable to load saved static model for test
                mode.', e);
                testStaticModel = null;
                testStaticLabels = [];
            }
        }
    }

    if (!testDynamicModel) {
        const savedDynamicLabels =
        localStorage.getItem(`${STORAGE_KEYS[currentLang].labels}-dynamic`);
        if (savedDynamicLabels) {
            testDynamicLabels = JSON.parse(savedDynamicLabels);
            try {
                testDynamicModel = await
                tf.loadLayersModel(`localstorage://${STORAGE_KEYS[currentLan
                g].model}-dynamic`);
            } catch (e) {
                console.warn('Unable to load saved dynamic model for test
                mode.', e);
                testDynamicModel = null;
                testDynamicLabels = [];
            }
        }
    }
}

function runStaticTestPrediction(flatLandmarks) {
    if (!testStaticModel || !testStaticLabels.length) {
        setTestResult('No static model available for testing.');
        return;
```

```
    }

    tf.tidy(() => {
        const input = tf.tensor2d([flatLandmarks]);
        const pred = testStaticModel.predict(input);
        const conf = pred.max().dataSync()[0];
        const idx = pred.argMax(-1).dataSync()[0];
        const label = testStaticLabels[idx] || 'Unknown';
        setTestResult(`Test (Static): ${label} (${Math.round(conf *
        100)}%)`);
    });
}

function runDynamicTestPrediction(flatLandmarks) {
    if (!testDynamicModel || !testDynamicLabels.length) {
        setTestResult('No dynamic model available for testing.');
        return;
    }

    if (testDynamicBufferStartTime === 0) {
        testDynamicBufferStartTime = Date.now();
    }

    testDynamicFrameBuffer.push(flatLandmarks);
    if (testDynamicFrameBuffer.length > MAX_DYNAMIC_FRAMES) {
        testDynamicFrameBuffer.shift();
    }

    const dynamicReady = (Date.now() - testDynamicBufferStartTime) >=
    TEST_DYNAMIC_ANALYZE_MS;
    if (!dynamicReady || testDynamicFrameBuffer.length < 1) {
        setTestResult(`Test (Dynamic): collecting frames
        ${testDynamicFrameBuffer.length}/${MAX_DYNAMIC_FRAMES}`);
        return;
    }
```

```
    tf.tidy(() => {
        const paddedFrames = [...testDynamicFrameBuffer];
        const lastFrame = paddedFrames[paddedFrames.length - 1];
        while (paddedFrames.length < MAX_DYNAMIC_FRAMES) {
            paddedFrames.push(lastFrame);
        }

        const input = tf.tensor3d([paddedFrames]);
        const pred = testDynamicModel.predict(input);
        const conf = pred.max().dataSync()[0];
        const idx = pred.argMax(-1).dataSync()[0];
        const label = testDynamicLabels[idx] || 'Unknown';
        setTestResult(`Test (Dynamic): ${label} (${Math.round(conf *
        100)}%)`);
    });
}

async function toggleTestMode() {
    if (isTestMode) {
        isTestMode = false;
        testDynamicFrameBuffer = [];
        testDynamicBufferStartTime = 0;
        if (testBtn) {
            testBtn.innerHTML = '<span class="material-
            icons">science</span>Start Test Mode';
            testBtn.classList.remove('primary-btn');
            testBtn.classList.add('secondary-btn');
        }
        setTestResult('Test mode is off.');
        return;
    }

    setTestResult('Loading models for test mode...');
    await loadTestModels();

    if (!testStaticModel && !testDynamicModel) {
```

```
        setTestResult('No trained/saved model found. Train first, then
        test.');
        alert('No trained/saved model found for testing. Train and save
        first.');
        return;
    }
```

A.3 Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>SignLink - Connect Beyond Words</title>
</head>
<body>

    <section class="hero-section">
        <!-- Radial Glow Background & Text -->
        <div class="cursor-light"></div>
        <div class="hero-text">
            SIGN LANGUAGE<br>AI INTERPRETER
        </div>

    <!-- Mobile Menu Drawer -->
    <div class="mobile-menu" id="mobileMenu">
        <a href="index.html">HOME</a>
        <a href="videocall.html">VIDEO CALL</a>
        <a href="translation.html">INTERPRETER</a>
        <a href="training.html">AI STUDIO</a>
        <div class="mobile-menu-footer">SIGNLINK · BREAKING THE SILENCE
        WITH AI.</div>
    </div>
```

```
<!-- Glass Navbar -->
<nav>
    <div>
        <div class="logo">SignLink</div>
        <div class="tagline">Breaking the Silence with AI.</div>
    </div>
    <div class="nav-links">
        <div class="nav-column">
            <a href="videocall.html">VIDEO CALL</a>
            <a href="translation.html">INTERPRETER</a>
            <a href="training.html">AI STUDIO</a>
        </div>
        <div class="nav-column">
            <span>INFO@SIGNLINK.AI</span>
            <span>+1 800 555 0199</span>
        </div>
    </div>
    <!-- Hamburger button → only visible on 1024px via CSS -->
    <button class="hamburger" id="hamburger" aria-label="Open menu">
        <span></span>
        <span></span>
        <span></span>
    </button>
</nav>


        <!-- 3D Gallery Container -->
        <div class="gallery-container">
            <div class="gallery" id="gallery">
                <!-- Floating wrapper for ambient motion -->
                <div class="gallery-float">
                    <!-- Left Item (Video Call) -->
                    <a href="videocall.html" class="gallery-item left"
                    data-index="0">
                        <div class="video-wrapper">
                            <video autoplay loop muted playsinline>
                                <source src="card-video-3.mp4"
```

```
                type="video/mp4">
            </video>
            <div class="video-title-overlay">
                <div class="video-subtitle">CONNECT /
                DETECT</div>
                <div class="video-title">VIDEO
                CALL</div>
            </div>
        </div>
    </a>

    <!-- Center Item (Main UI / Interpreter) -->

    <a href="translation.html" class="gallery-item
    center" data-index="1">
        <div class="video-wrapper">
            <video autoplay loop muted playsinline>
                <source src="card-video-1.mp4"
                type="video/mp4">
            </video>
            <div class="video-title-overlay">
                <div class="video-subtitle">TRANSLATION
                ENGINE</div>
                <div class="video-title">AI
                INTERPRETER</div>
            </div>
        </div>
    </a>

    <!-- Right Item (AI Training) -->
    <a href="training.html" class="gallery-item right"
    data-index="2">
        <div class="video-wrapper">
            <video autoplay loop muted playsinline>
                <source src="card-video-2.mp4"
                type="video/mp4">
```

```
                            </video>
                            <div class="video-title-overlay">
                                <div class="video-subtitle">CUSTOMIZE /
                                LEARN</div>
                                <div class="video-title">AI STUDIO</div>
                            </div>
                        </div>
                    </a>
                </div>
            </div>
        </div>

        <!-- Active Indicator Bar -->
        <div class="controls-container" id="controls">
            <div class="control-dot"></div>
            <div class="control-dot active"></div>
            <div class="control-dot"></div>
        </div>
</section>

<!-- Features Section -->
<section class="features-section">
    <span class="section-tag">Empowering Communication</span>
    <h2 class="section-title">The most advanced <em>AI engine</em> for
    Sign Language.</h2>

    <div class="features-grid">
        <div class="feature-card">
            <h3>Real-time Translation</h3>
            <p>Proprietary neural networks convert sign language to
            text and speech instantly, breaking barriers in every
            conversation.</p>
        </div>
        <div class="feature-card">
            <h3>Infinite Dataset</h3>
            <p>Our AI Studio allows users to contribute new signs,
```

```
            continuously expanding the linguistic reach of the platform.
            </p>
        </div>
        <div class="feature-card">
            <h3>Cross-Platform</h3>
            <p>WebRTC-powered calls and mobile-first design ensure
            SignLink is available whenever and wherever you need it.</p>
        </div>
    </div>
</section>

<!-- Footer -->
<footer>
    <div class="footer-grid">
        <div>
            <div class="footer-logo">SIGNLINK</div>
            <p class="footer-tagline">Giving every hand a voice.
            Breaking the silence with artificial intelligence.</p>
        </div>
        <div>
            <div class="footer-head">Navigation</div>
            <ul class="footer-links">
                <li><a href="index.html">Home</a></li>
                <li><a href="translation.html">Interpreter</a></li>
                <li><a href="videocall.html">Video Call</a></li>
                <li><a href="training.html">AI Studio</a></li>
            </ul>
        </div>
        <div>
            <div class="footer-head">Contact & Social</div>
            <ul class="footer-links">
                <li><a href="mailto:support@signlink.ai">Email
                Support</a></li>
                <li><a href="#">GitHub Repository</a></li>
                <li><a href="#">LinkedIn</a></li>
                <li><a href="#">X / Twitter</a></li>
```

```
                </ul>
            </div>
        </div>
        <div class="footer-bottom">
            <span>© 2026 SIGNLINK PROJECT. ALL RIGHTS RESERVED.</span>
            <span>BUILT WITH MEDIAPIPE • TENSORFLOW • WEBRTC</span>
        </div>
    </footer>
```

Appendix B

Screenshots

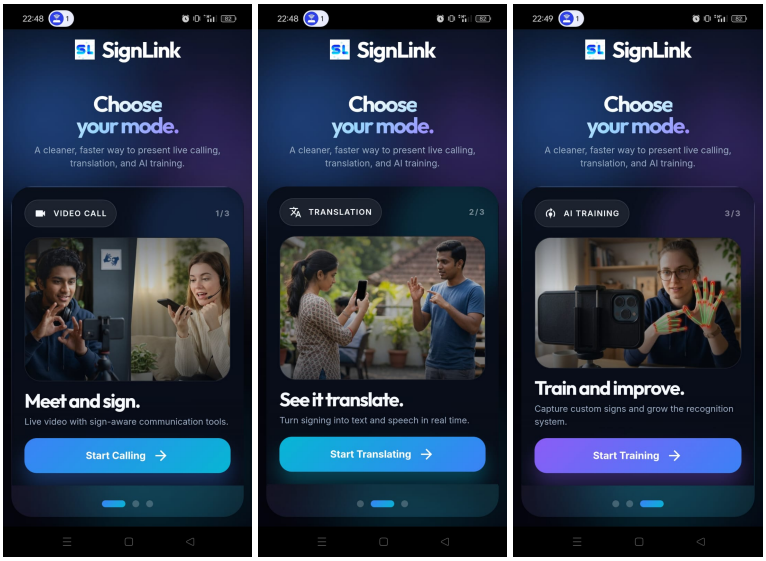

Figure B.1: Home Page

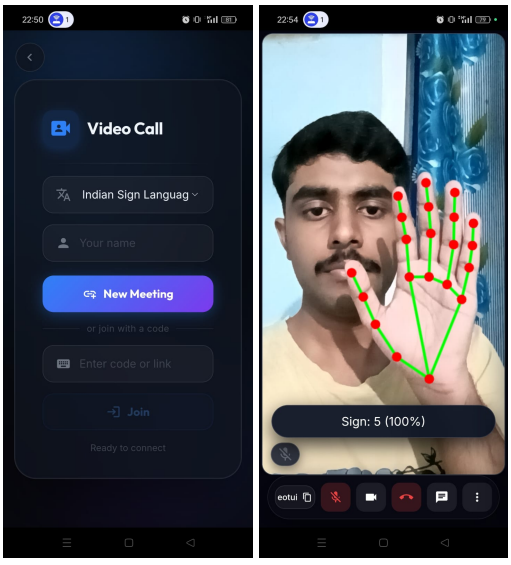

Figure B.2: Video Call Page

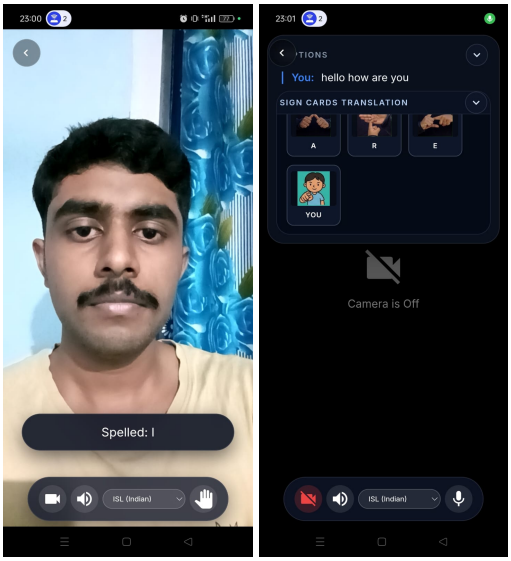

Figure B.3: Live Translation Page

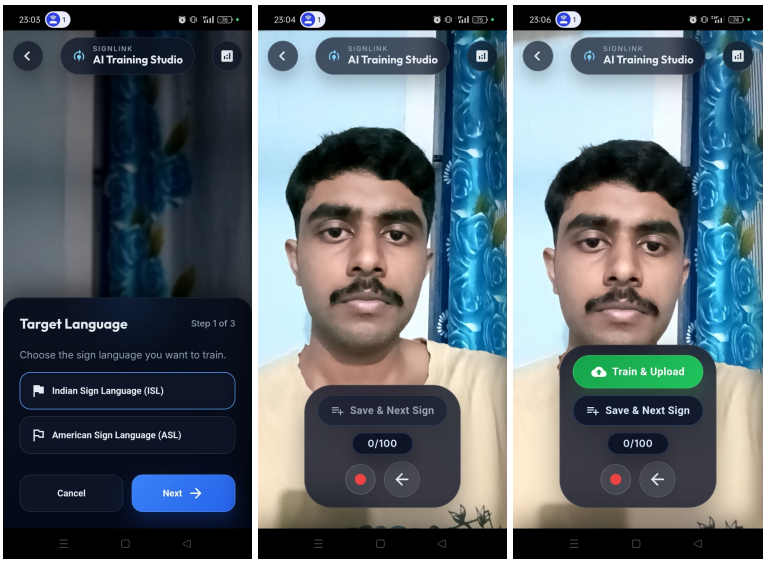

Figure B.4: AI Training Page